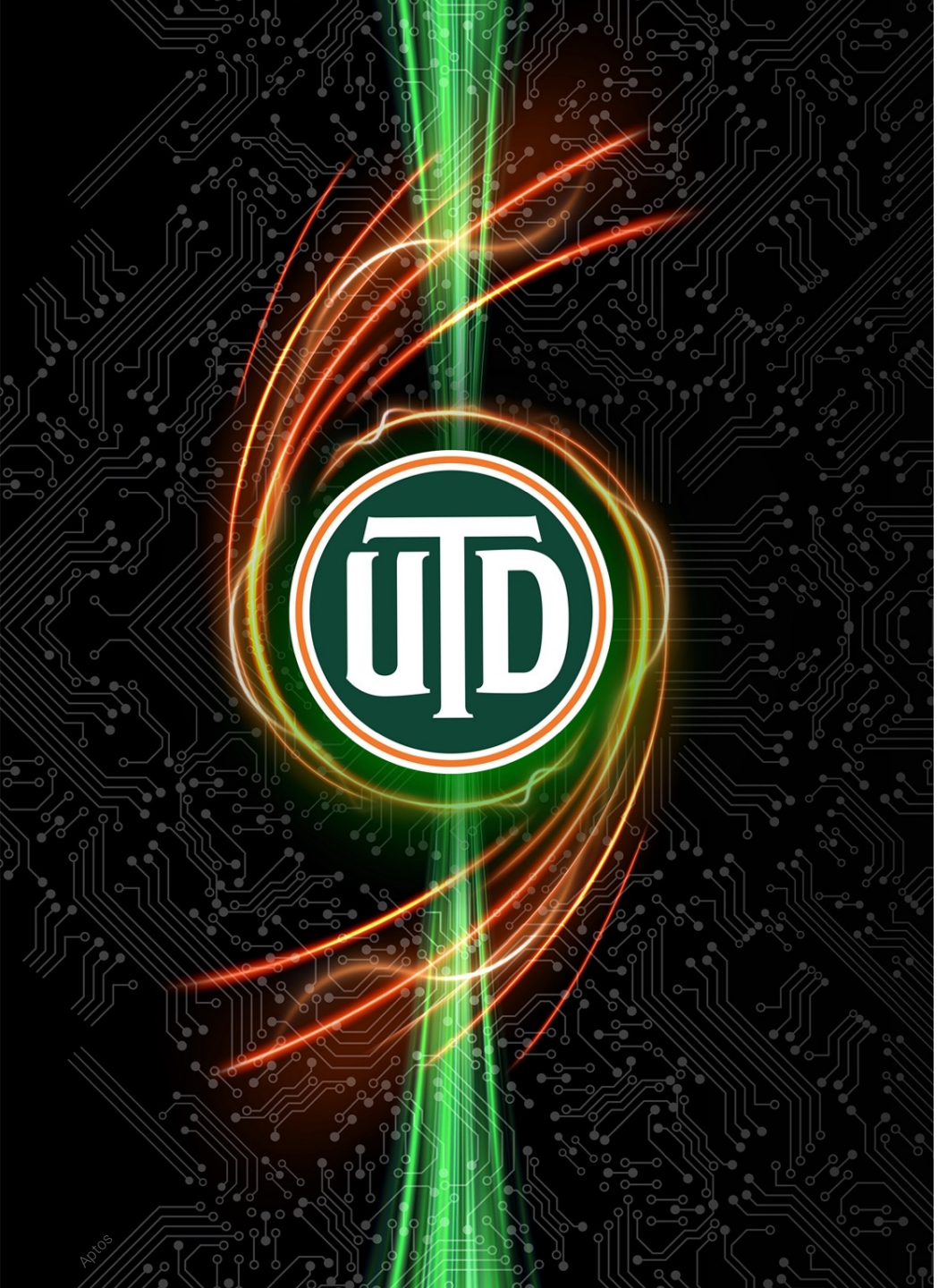
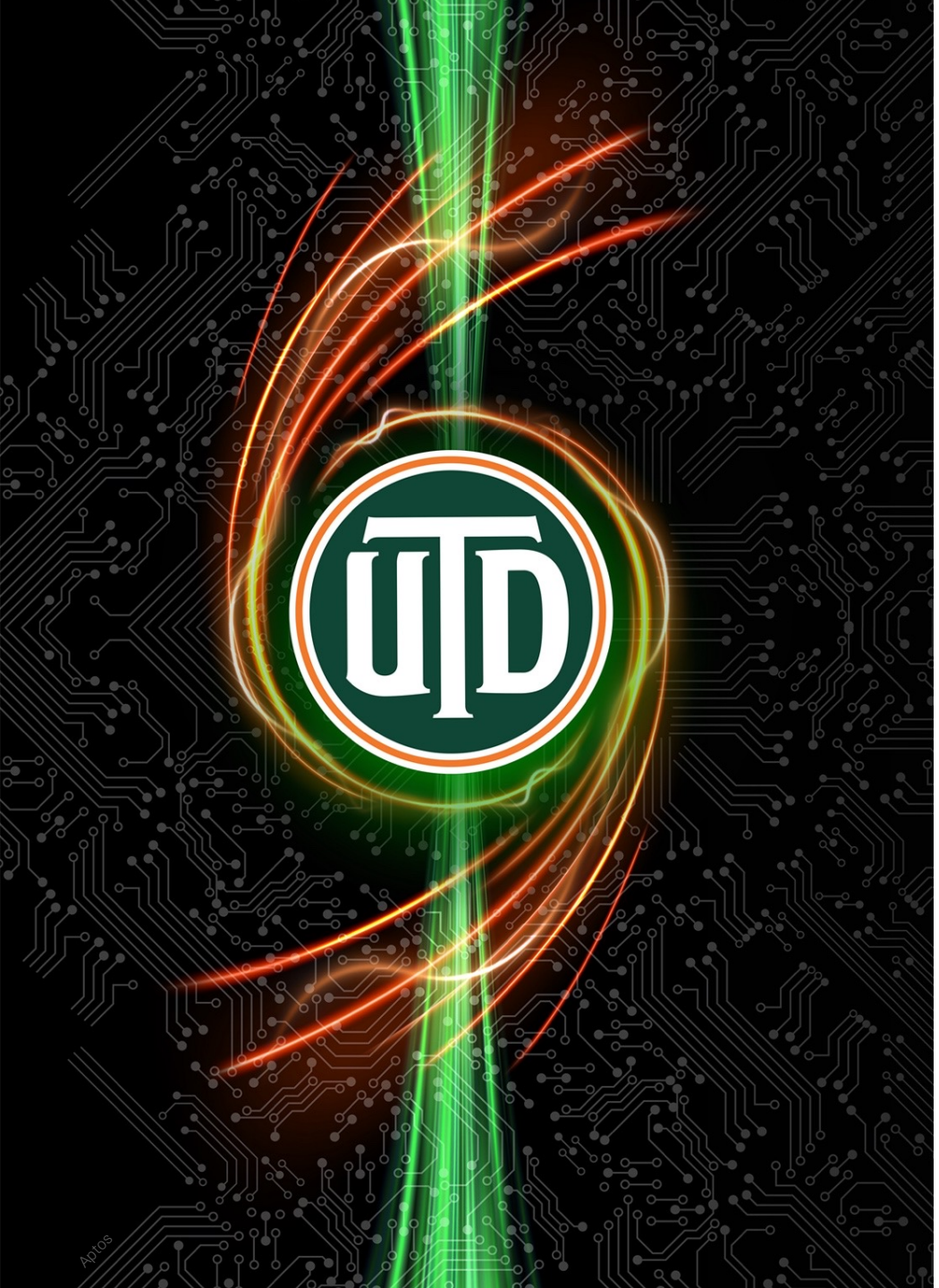


The latest version of this presentation is available at https://hpc.utdallas.edu/systems-resources/juno/.
Please always use the latest version for reference.

Juno Orientation

Summer 2026 v14, July 2026

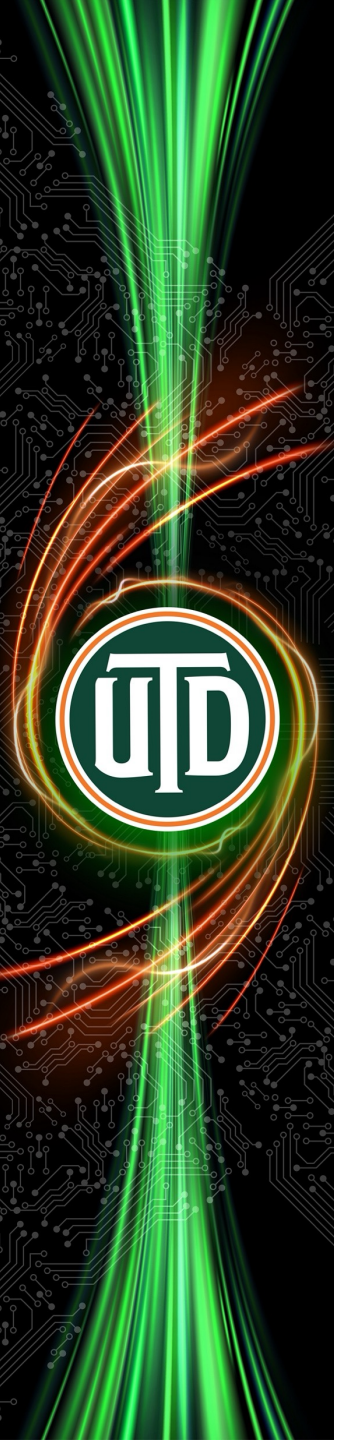
HPC@UTD



Juno Orientation

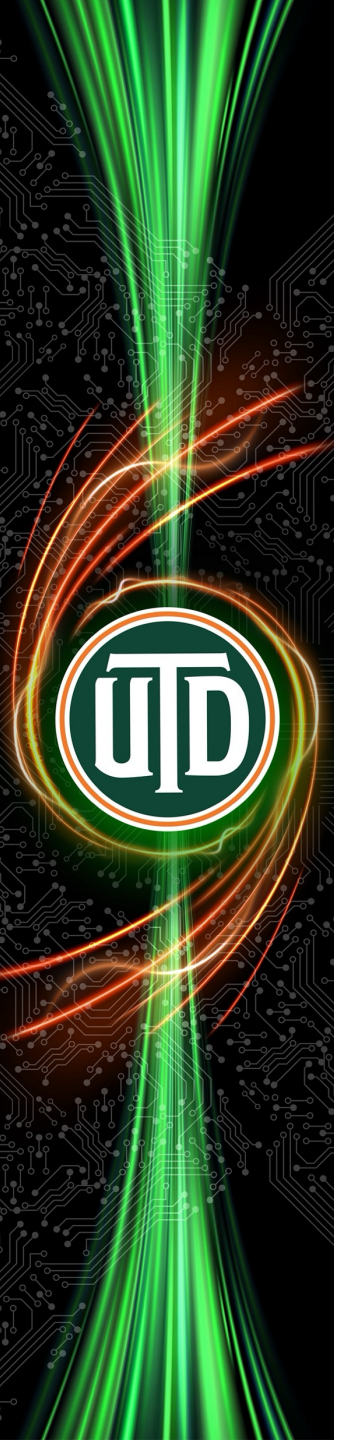
Summer 2026 v14, July 2026

HPC@UTD



Outline

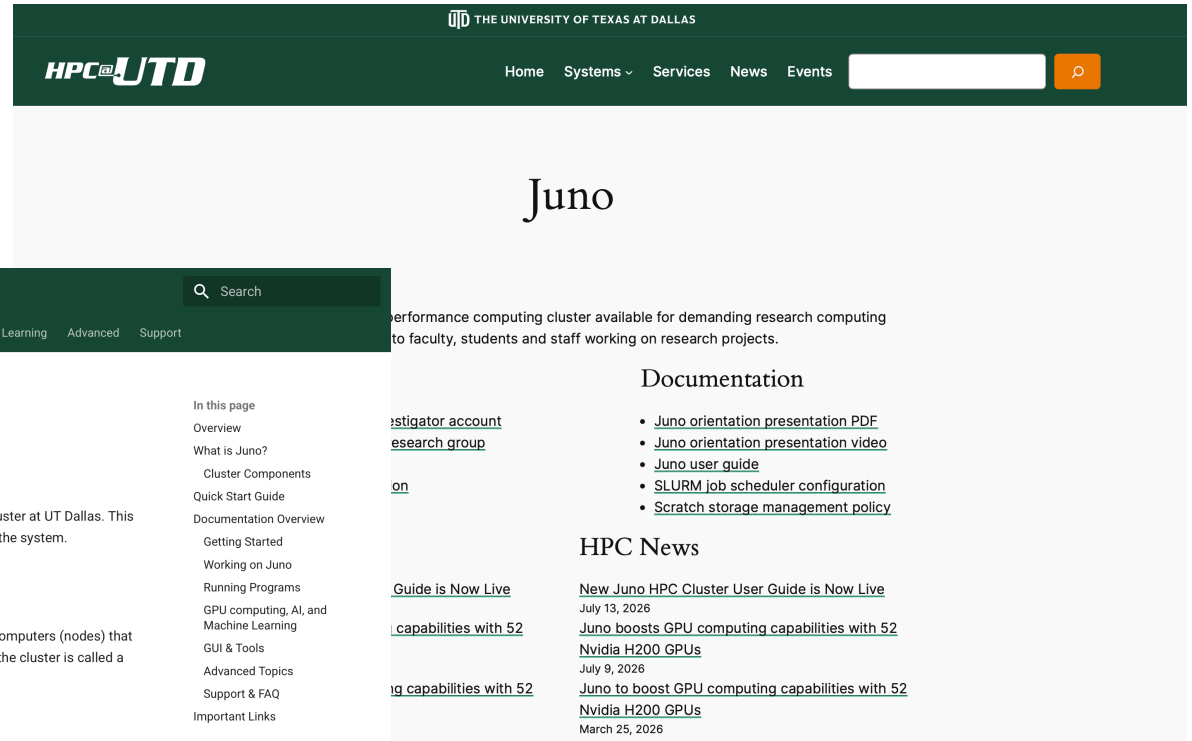
- Getting help
- Juno is an HPC cluster
- Directories and data management
- Basic Unix commands summary
- Logging into Juno and transferring data
- Introduction to the SLURM job scheduler
- Achieving high performance using parallel processing

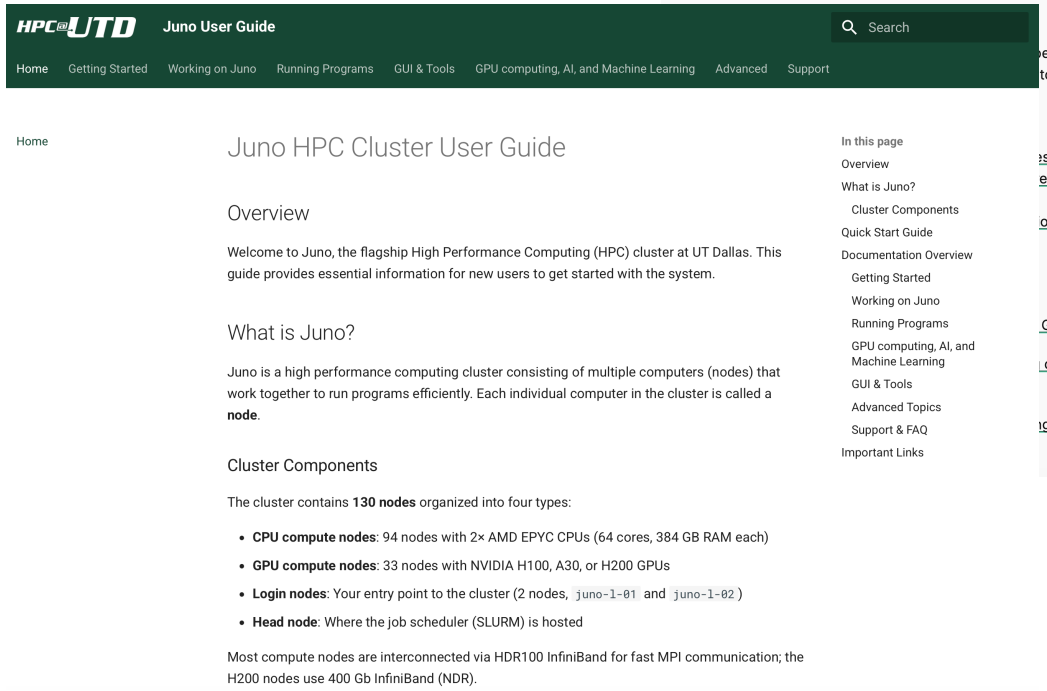


Getting help

Available resources

- Juno page
- Juno user guide





Getting help

1. Please visit the HPC website's services page https://hpc.utdallas.edu/services/
2. Locate your area/system
   - HPC services
   - Juno
   - Ganymede2
   - HPC storage
3. Click on the link that most closely matches your needs

- When unsure, please email circ-assist@utdallas.edu

Services

We provide the research community guidance and cutting-edge high performance computing resources and services.

To request assistance, please open a ticket using the link corresponding to the your system and service here that most closely matches your current needs. You can also contact us via email at circ-assist@utdallas.edu or visit our office on weekdays between 9:30 a.m. and 3:30 p.m. in the Administration Building AD 3.207.

HPC Services

HPC Facilitation Services provide personalized support to help researchers make the most of HPC resources.

- Consultation on using HPC systems and services
- Orientation training and onboarding service
- Assistance with code porting
- Consultation on parallel programming
- General user support

Juno HPC System

Juno is UTD's flagship high performance computing cluster available to the UTD community for research computing workloads.

- Add a new PI account
- Add new account for research group member
- General user support
- Software installation request

Ganymede

Ganymede is UTD's oldest high performance computing cluster, retiring on April 10, 2026.

- General user support

Ganymede 2 HPC System

Ganymede 2 is UTD's high performance computing "condo" cluster. Faculty can buy nodes for Ganymede 2 for their priority use but other users can also use the cluster.

- Add a new PI account
- Add new account for research group member
- General user support
- Software installation request
- Consultation for Ganymede 2 condo nodes

IO HPC Storage

The "IO" HPC storage system hosts the home and /groups directories of Juno and Ganymede 2 users.

- Request new storage allocation
- Increase storage allocation
- Data migration service
- General storage user support
- General consultation on storage

Titan Research Data Condo Storage

Titan stores large volumes of research data using a "purchase plus maintenance cost" model for the life of the unit. Storage is mounted on Juno and Ganymede 2 login nodes, and on researcher laptops and workstations.

(More information is coming soon.)

Saturn Research Data Storage

Saturn research data storage system offers enterprise-grade security, resilience, redundancy, and uptime, as a centrally managed research data storage service designed to support research activity. Learn more...

(Saturn is managed by a joint OIT/ORI team, not the HPC team.)

Consultation on acquiring hardware

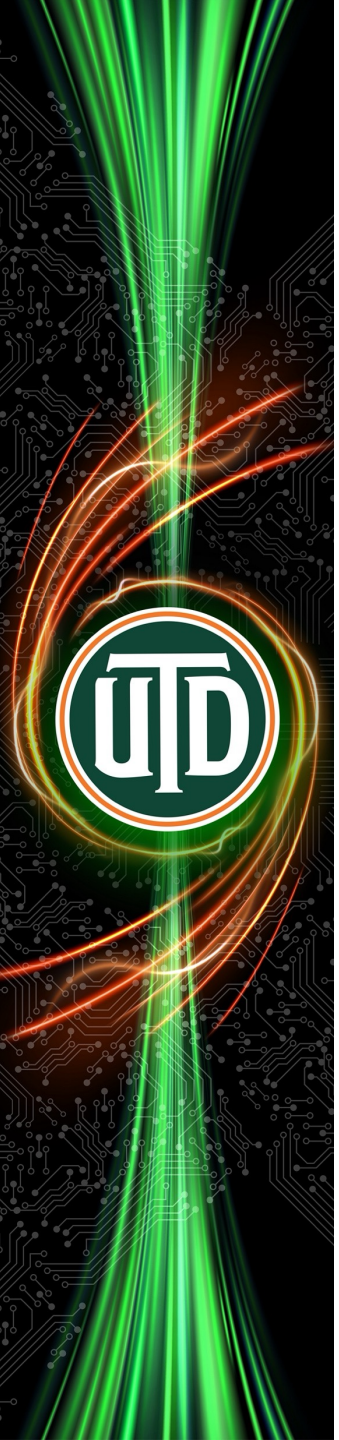
The HPC team can tell you about available high performance computing options available to UTD faculty, and help you choose sustainable options that meet your needs.

- Request hardware acquisition consultation

Consultation on new research computing initiatives

Broad consultation and guidance for faculty, departments, centers and schools considering new research computing initiatives.

- Request consultation on new research computing initiatives



Getting help

Services

We provide the research community guidance and cutting-edge high performance computing resources and services.

To request assistance, please open a ticket using the link corresponding to the your system and service here that most closely matches your current needs. You can also contact us via email at circ-assist@utdallas.edu or visit our office on weekdays between 9:30 a.m. and 3:30 p.m. in the Administration Building AD 3.207.

HPC Services

HPC Facilitation Services provide personalized support to help researchers make the most of HPC resources.

- Consultation on using HPC systems and services
- Orientation training and onboarding service
- Assistance with code porting
- Consultation on parallel programming
- General user support

Juno HPC System

Juno is UTD's flagship high performance computing cluster available to the UTD community for research computing workloads.

- Add a new PI account
- Add new account for research group member
- General user support
- Software installation request

Ganymede

Ganymede is UTD's oldest high performance computing cluster, retiring on April 10, 2026.

- General user support

Ganymede 2 HPC System

Ganymede 2 is UTD's high performance computing "condo" cluster. Faculty can buy nodes for Ganymede 2 for their priority use but other users can also use the cluster.

- Add a new PI account
- Add new account for research group member
- General user support
- Software installation request
- Consultation for Ganymede 2 condo nodes

IO HPC Storage

The "IO" HPC storage system hosts the home and /groups directories of Juno and Ganymede 2 users.

- Request new storage allocation
- Increase storage allocation
- Data migration service
- General storage user support
- General consultation on storage

Titan Research Data Condo Storage

Titan stores large volumes of research data using a "purchase plus maintenance cost" model for the life of the unit. Storage is mounted on Juno and Ganymede 2 login nodes, and on researcher laptops and workstations.

(More information is coming soon.)

Saturn Research Data Storage

Saturn research data storage system offers enterprise-grade security, resilience, redundancy, and uptime, as a centrally managed research data storage service designed to support research activity. Learn more...

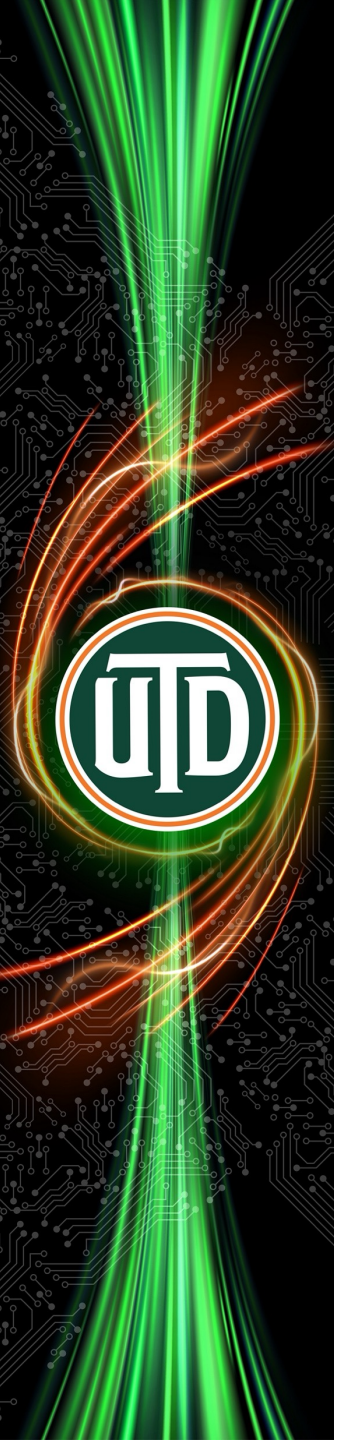
(Saturn is managed by a joint OIT/ORI team, not the HPC team.)

Consultation on acquiring hardware

The HPC team can tell you about available high performance computing options available to UTD faculty, and help you choose sustainable options that

Consultation on new research computing initiatives

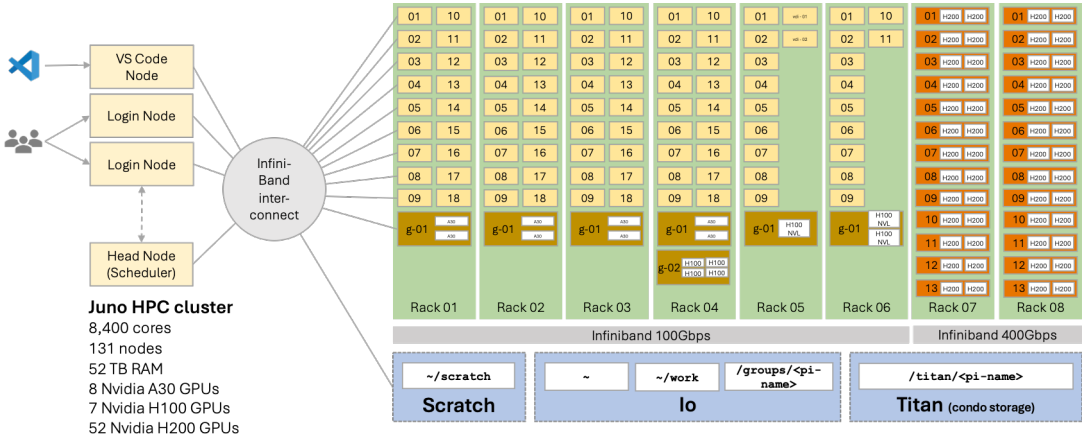
Broad consultation and guidance for faculty, departments, centers and schools considering new



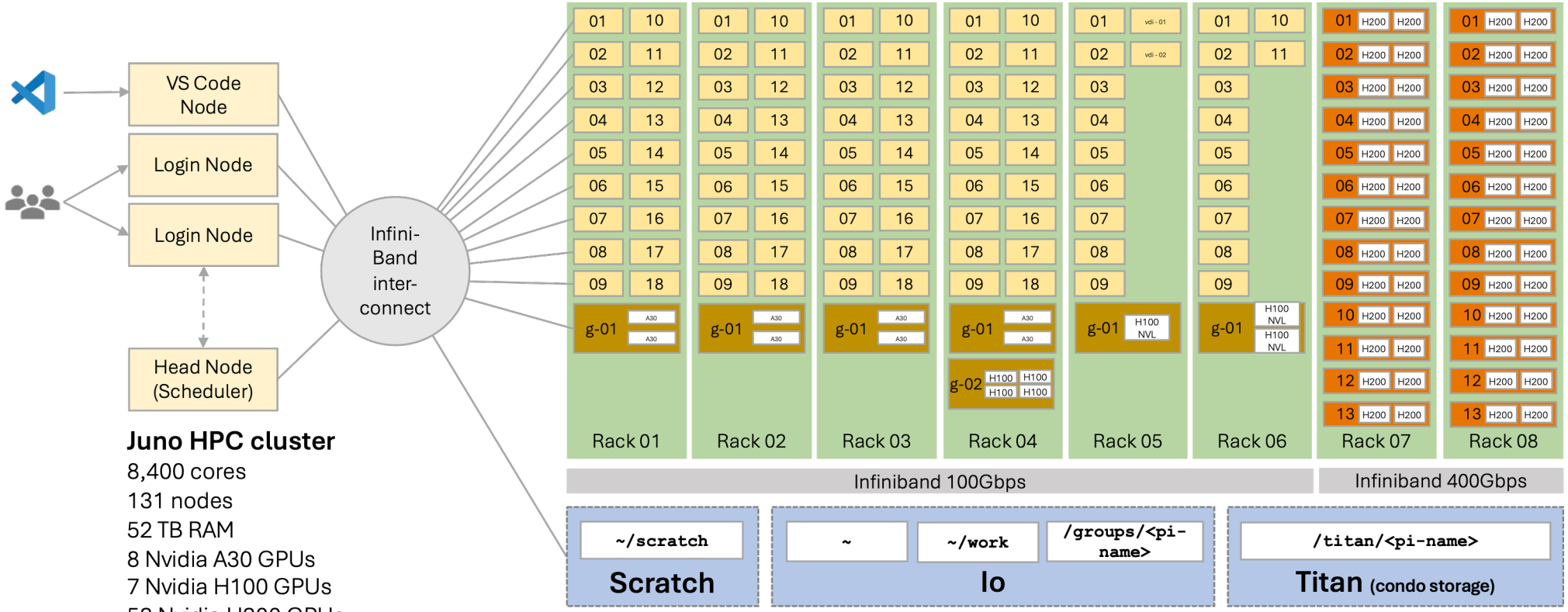
Juno is an HPC cluster

Juno is an HPC cluster

- An HPC cluster consists of several computers that work together to run programs very fast.
  - Each individual computer is called a node.
- HPC cluster nodes have different roles. These are:
  - Compute nodes: CPU nodes, GPU nodes
  - Login nodes
  - Head (or scheduler) nodes
- HPC clusters also have access to high performance storage systems
  - Storage holds your data and programs
- Nodes are interconnected using high speed networks



Juno’s configuration



Juno HPC cluster
8,400 cores
131 nodes
52 TB RAM
8 Nvidia A30 GPUs
7 Nvidia H100 GPUs
52 Nvidia H200 GPUs

HPC clusters are not workstations/servers

✓ Shared computing resource

✓ Scheduler-driven

✓ Designed for **large** and **parallel** workloads

✓ Accelerating science and engineering discovery

✗ Not a personal workstation, no GUI, no **sudo**

✗ Not always on or exclusive

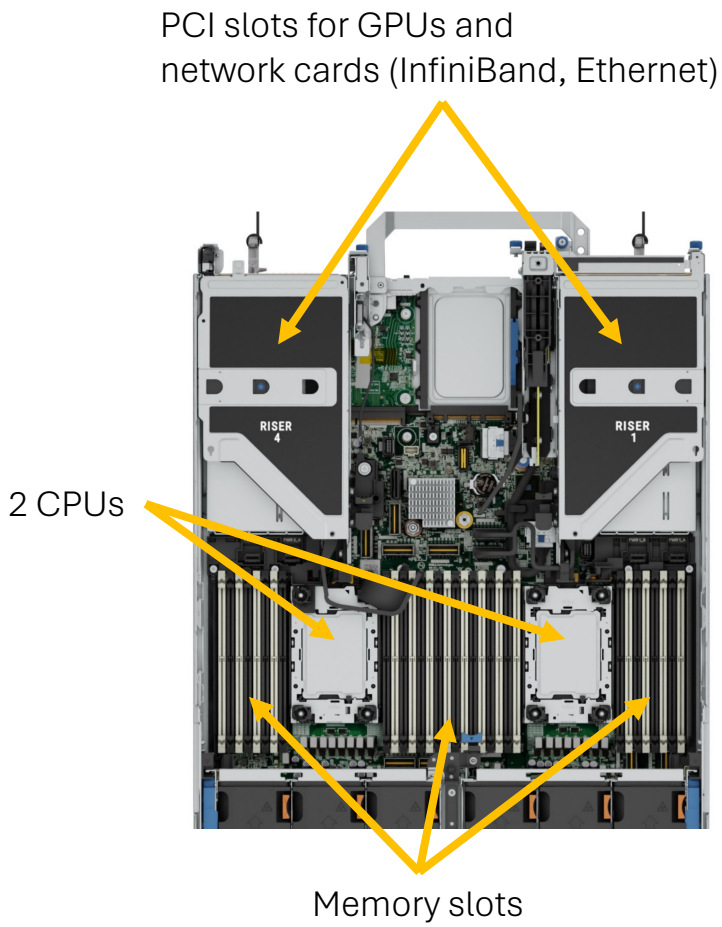
✗ Not optimized for small, serial, always-running tasks

✗ Not a general-purpose server (web hosting, databases, crypto mining, AI inference etc.)

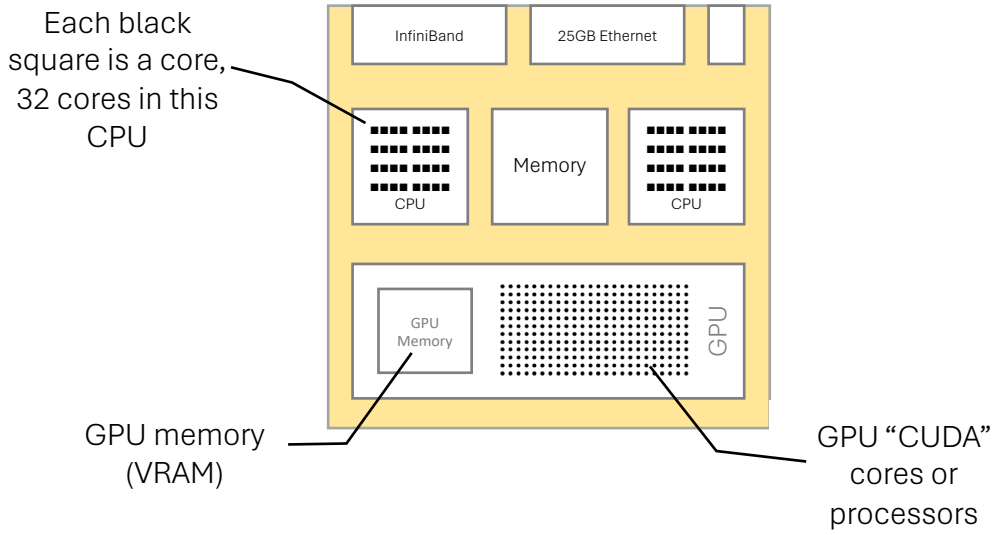
✓ We assume that you already have an established workflow and you need more computing demands

✗ I need a computer as my laptop is running out of space

Terminology: CPUs, cores, VRAM, GPU cores

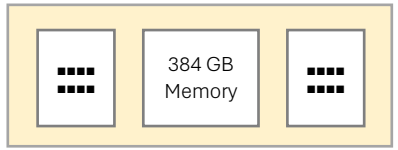


- CPU refers to a chip that installs in a motherboard socket
  - A CPU contains several cores
- Core refers to a processor capable of running a program

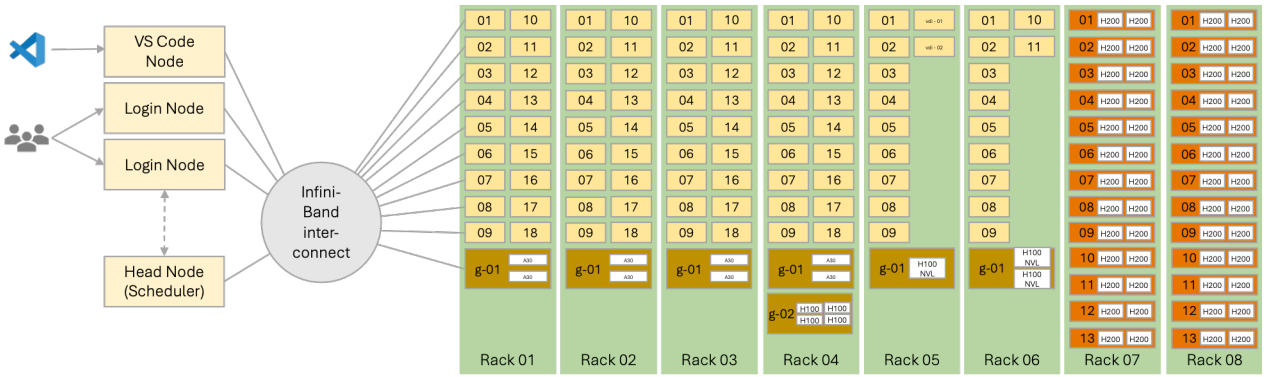


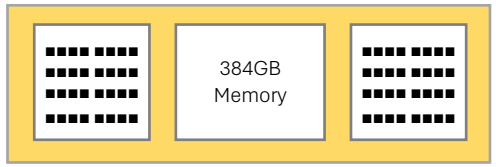
Source: https://www.delltechnologies.com/asset/en-us/products/servers/technical-support/poweredge-r760xa-technical-guide.pdf

Juno’s node configuration

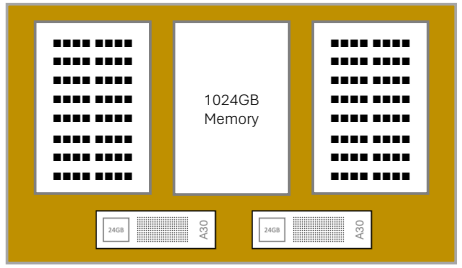


Login and Head Nodes (3)
2x Intel Xeon Sliver 4309Y 2.8GHz 8C/16T 128MB cache
128 or 384 GB RAM (8x16GB), 3.84TB (2x1.92TB SSD)
HDR100 InfiniBand, 2x1Gbps Ethernet

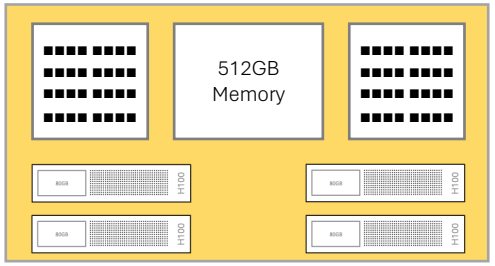




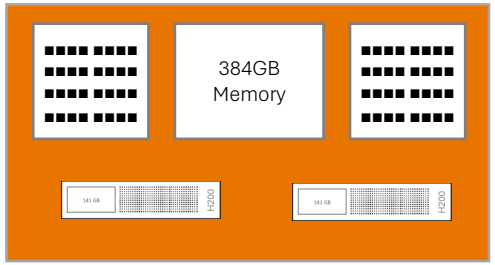
CPU Compute Nodes (94)
2x AMD EPYC 9334 2.7GHz 32C/32T 128M cache
384GB RAM (12x32GB), 480GB SSD
HDR100 InfiniBand, 10/25Gbps Ethernet



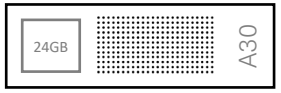
GPU Compute Nodes (4x2A30)
2x AMD EPYC 9534 2.45GHz 64C/64T 256M cache
2x Nvidia A30 24GB GPUs
1024GB RAM (16x64GB), 480GB SSD
HDR100 InfiniBand, 2x10/25Gbps Ethernet, 2x1Gbps Ethernet

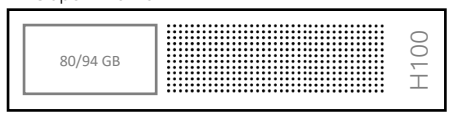


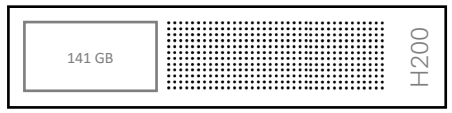
GPU Compute Nodes (1x4H100, 1x2H100, 1xH100)
2x Intel Xeon Platinum 8462Y 2.8GHz 32C/32T 60M cache
4x Nvidia HGX H100 80GB HBM3 GPUs, 2x Nvidia H100 94GB NVL GPUs, 1x Nvidia H100 94GB NVL GPU
512 GB RAM (32x16GB), 480GB SSD
HDR100 InfiniBand, 2x10/25Gbps Ethernet, 2x1Gbps Ethernet



GPU Compute Nodes (26x2H200)
2x AMD EPYC 9335 3.0GHz 32C/32T 128M cache
2x Nvidia H200 141GB NVL GPU
384 GB RAM (32x12GB), 480GB SSD
NDR400 InfiniBand, 2x10/25Gbps Ethernet, 2x1Gbps Ethernet

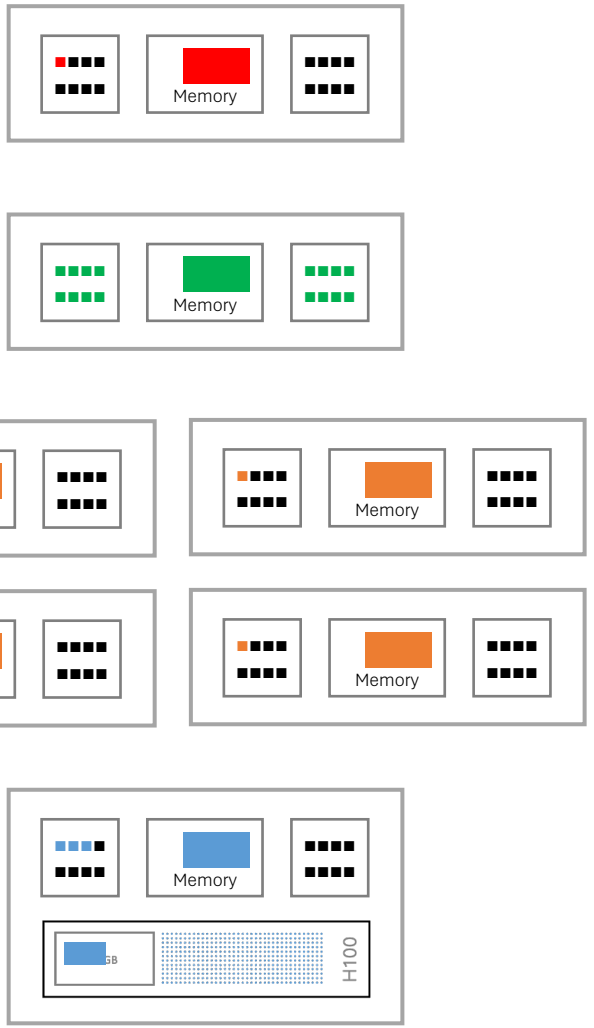


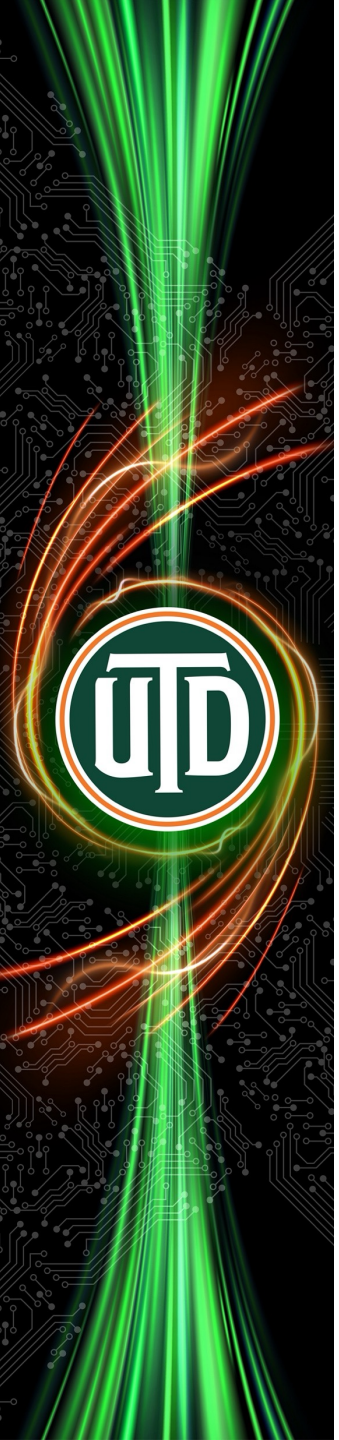




Achieving high performance on an HPC cluster

- Programs normally perform their work on just one core
  - This is called serial programs
- Achieving high performance requires the program to split their work and perform it simultaneously on multiple cores
  - This is called parallel programs
- On an HPC cluster, parallel execution can happen in 3 ways
  - Use multiple cores in single node (shared memory multiprocessor (SMP) or multithreaded parallel)
    - Using OpenMP or pthreads
  - Use cores in multiple nodes (message passing or distributed memory parallel)
    - Using MPI
  - Use the tiny "processors" in GPUs (GPU accelerated)
    - Using CUDA or OpenMP for GPUs

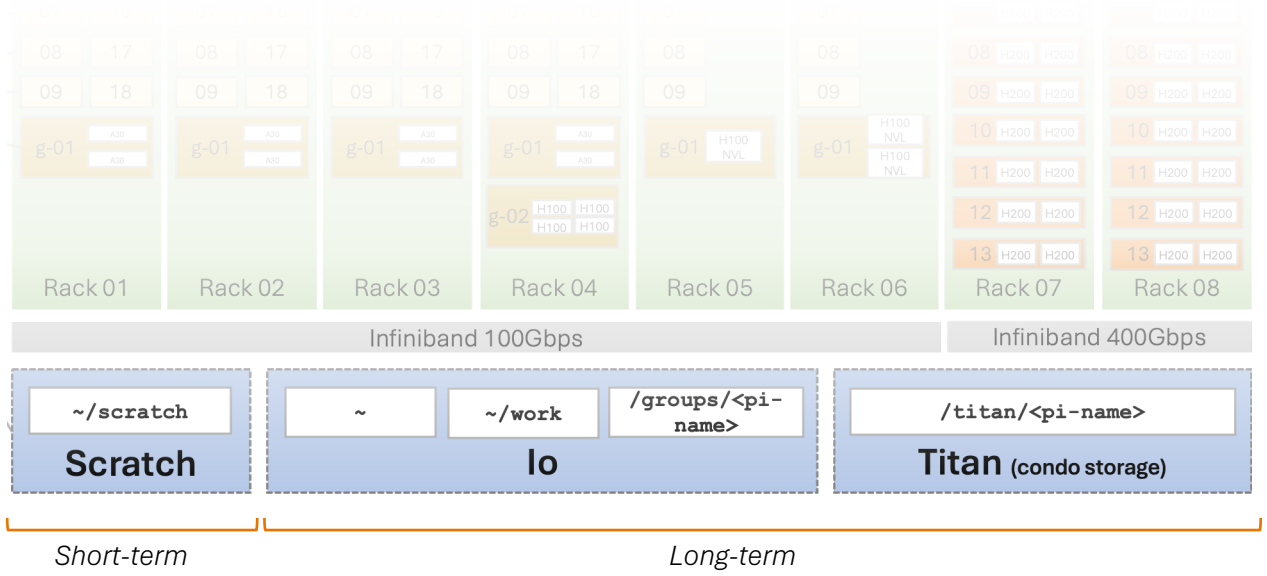




Directories and data management

HPC storage systems

- **Scratch** is the very high-performance storage system for high-speed IO during batch jobs
- **Io** is the high-speed storage system for storing programs and data used actively computations.
- **Titan** is an additional high-speed storage system for storing programs and data available through a condo model



Data directories on Io

Home	Work	Group
`~`	`~/work`	`/groups/<pi-name>`
50GB Private to user, backed up daily	1TB Private to user, backed up daily	1TB or more Shared by the research group, backed up daily
Use for configuration files, login scripts, user installed software, small input and output files	Use for user installed software, large software packages, large data files, results files	Use for group software and large datasets, models and results files
Do not use for batch jobs IO needs	Can use for batch jobs with light to moderate IO needs • Use Scratch space for jobs requiring heavy IO	

Tracking your storage usage on Io

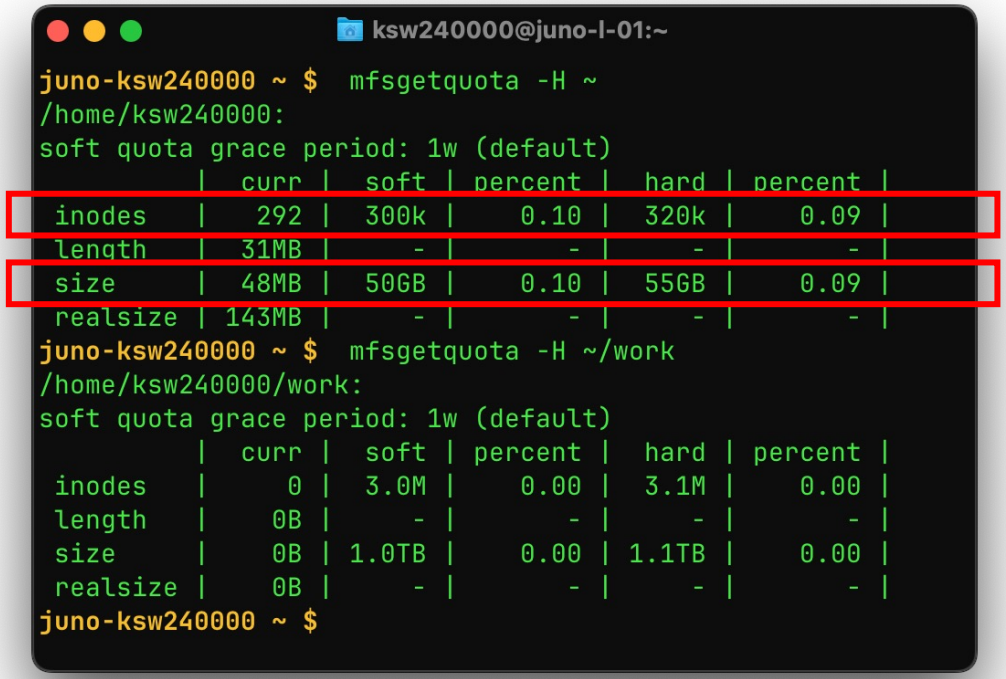
- Get your quota and usage using the command

  `mfsgetquota -H <directory>`

  e.g.,

  `mfsgetquota -H ~/work`

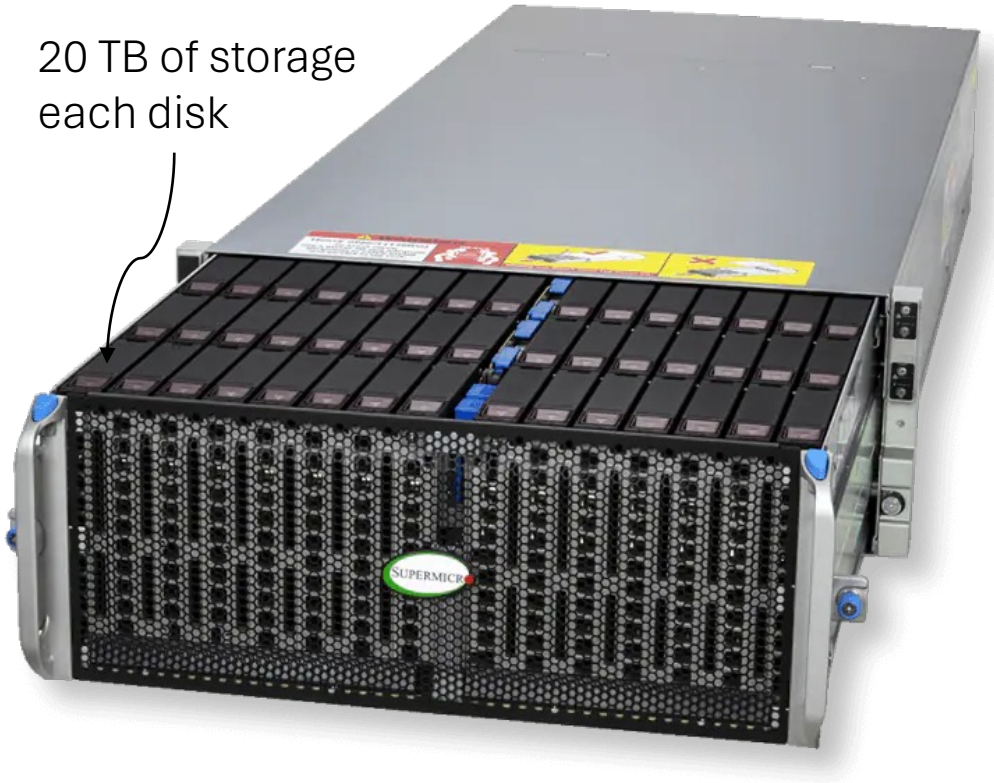
- Two kinds of quotas are implemented
  - Storage amount
  - Number of inodes (files and directories)



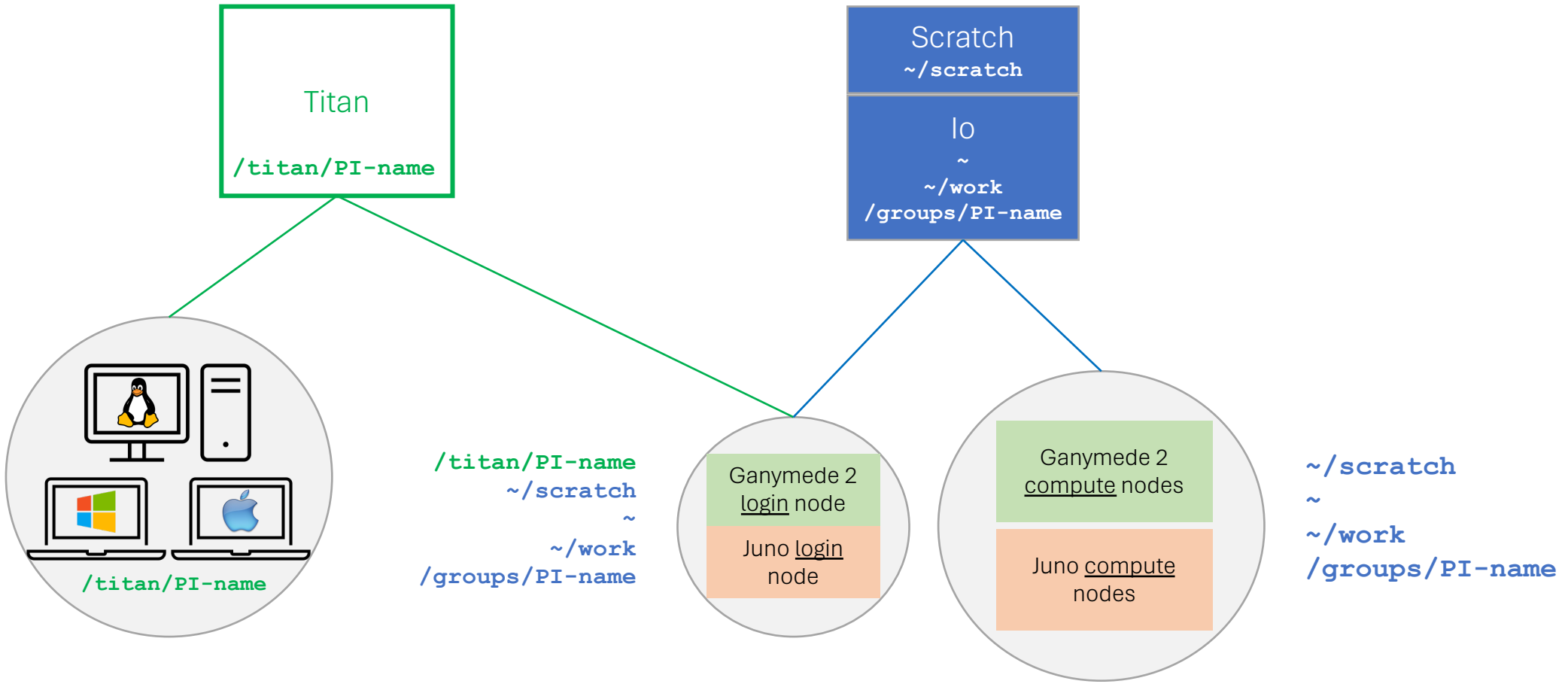
Titan condo storage

`/titan/<pi-name>`

- This research storage condo allows researchers to purchase storage as needed and share it with authorized users
- Start from 40 TB, with 20 TB increments
- More information at: https://hpc.utdallas.edu/guidelines/titan-condo-storage-customer-guidelines/

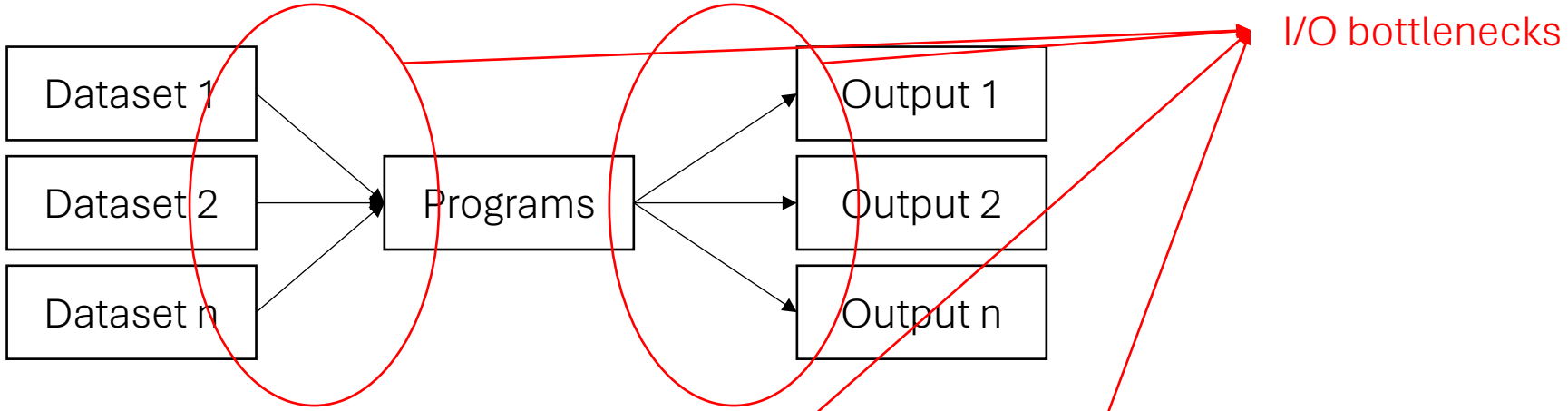


Titan condo storage system

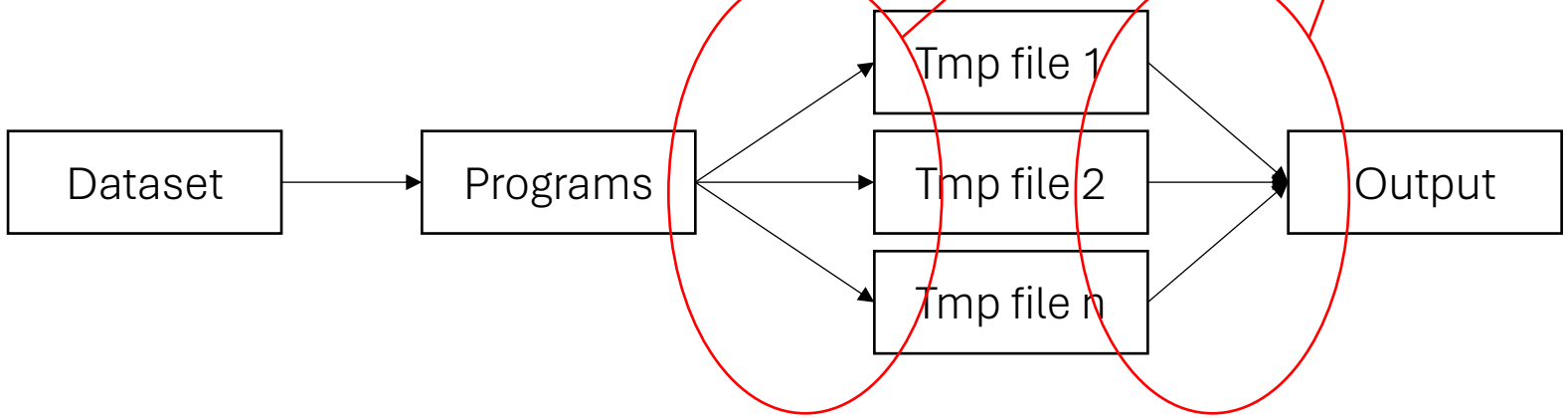


I/O needs for HPC programs

Scenario 1 – Intense Input/Output: for example, training a LLM model



Scenario 2 – Intermediate results: for example, running a DFT calculation



Scratch space

Scratch

~/scratch

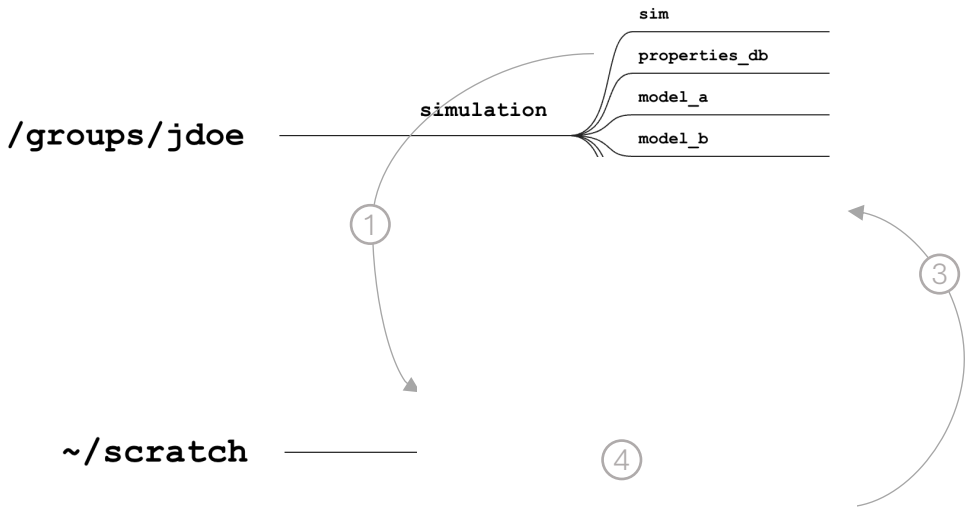
30TB (soft limit)

Private to user, never backed up, purged regularly

Use for I/O in batch jobs
Scratch is up to 10x faster at large I/O than **~**, **~/work** or **/groups**

Data movement to Scratch occurs in the following sequence:

1. **Copy** data **in** at the start
2. **Read** from and **write** to Scratch **during** the job
3. **Copy** data **out** at the end
4. **Clean up** before leaving



Scratch space

Scratch

~/scratch

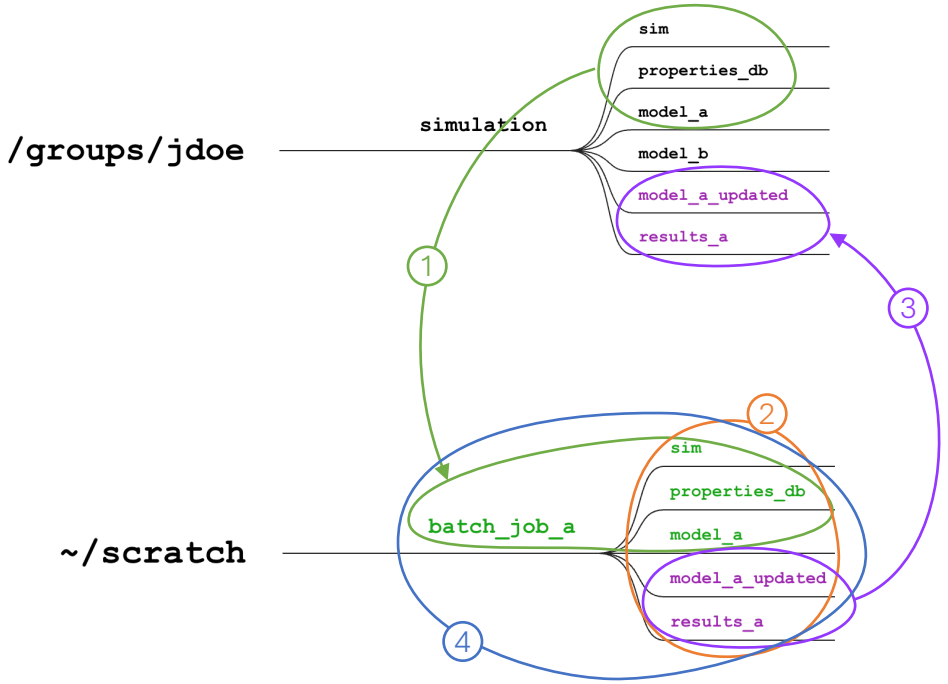
30TB (soft limit)

Private to user, never backed up, purged regularly

Use for I/O in batch jobs
Scratch is up to 10x faster at large I/O than **~**, **~/work** or **/groups**

Data movement to Scratch occurs in the following sequence:

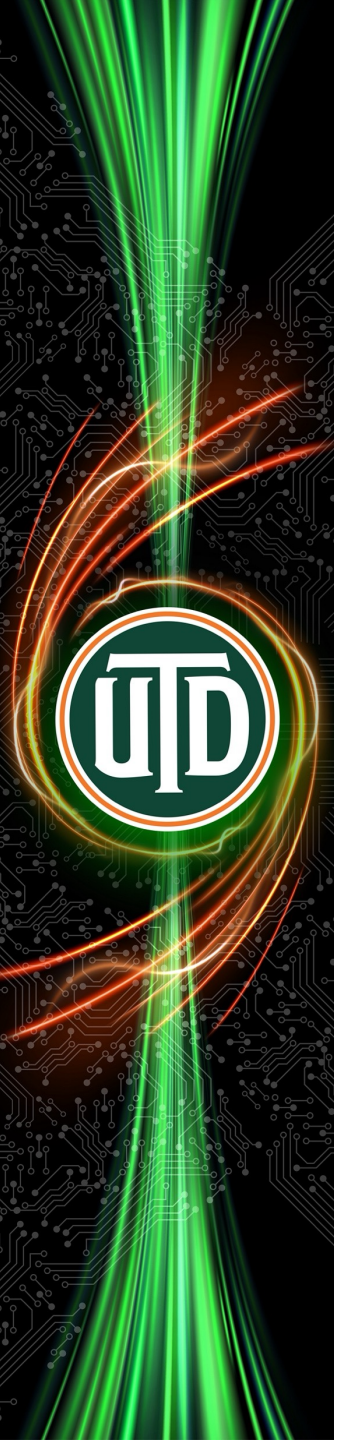
1. **Copy** data **in** at the start
2. **Read** from and **write** to Scratch **during** the job
3. **Copy** data **out** at the end
4. **Clean up** before leaving



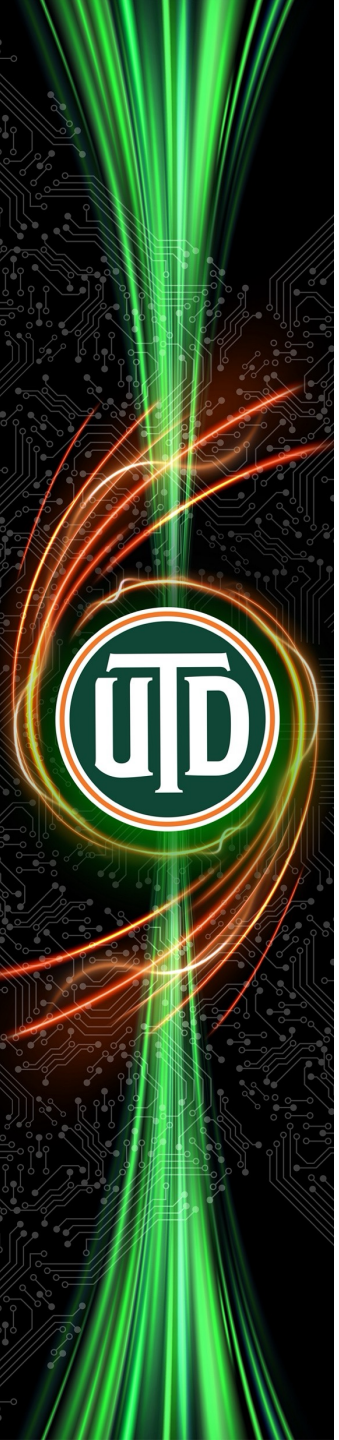
Scratch storage management policy

- Overall Scratch space is not very large, so we need to keep it clean
  - I.e., use it during batch jobs, delete afterwards
  - We regularly purge (delete old files from) Scratch space
- Scratch storage management policy (https://hpc.utdallas.edu/policies-and-guidelines/) states

> HPC Scratch storage (**`~/scratch`** directory) is for use during a job's execution. Batch jobs should utilize Scratch as "*copy in, read and write during, copy out, and clean up when done*" data space.
>
> Data stored in Scratch is never backed up. Scratch will be purged regularly to maintain sufficient space for new batch jobs to run. When purged, any file or directory not accessed for 45 days will be deleted. The HPC staff will monitor Scratch utilization and, in the future, may reduce the (currently) 45 day time limit.

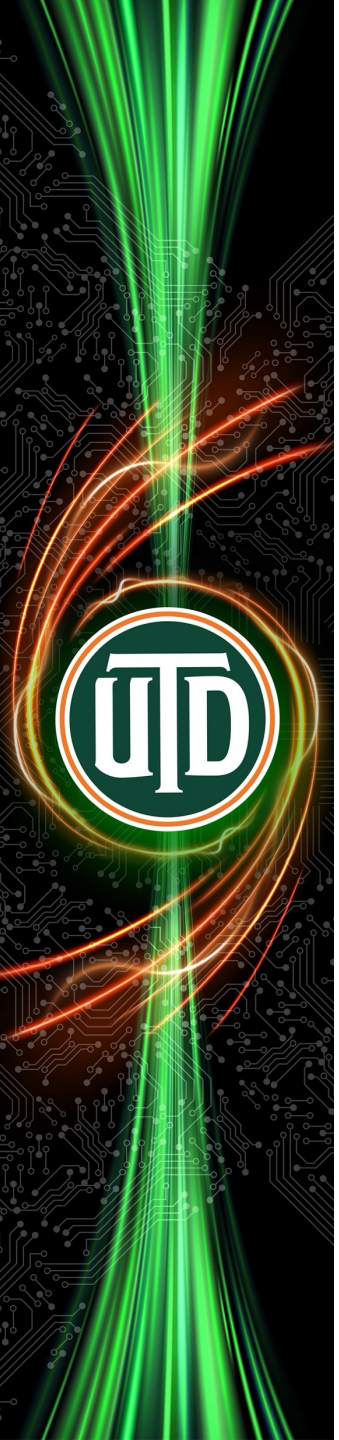


Basic Unix commands



Basic Unix commands (not covered here)

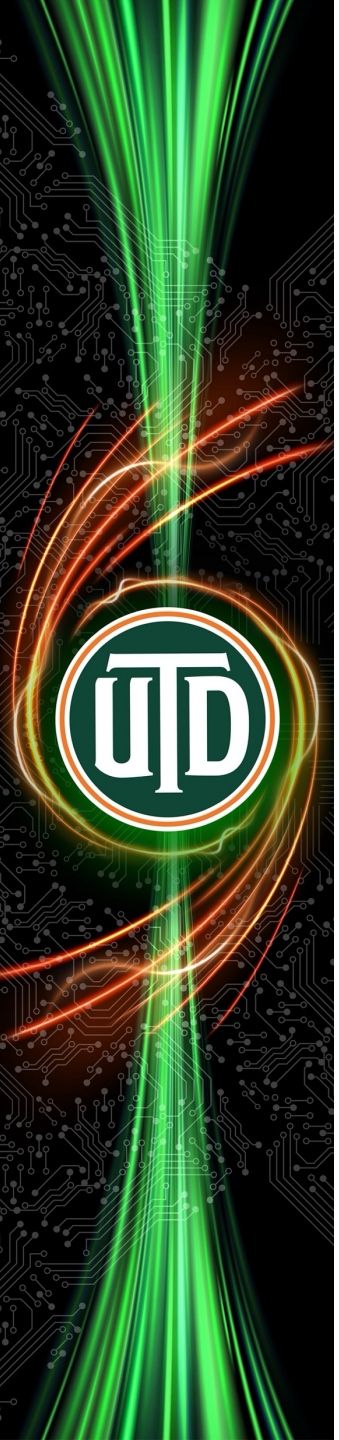
- Directories
  - `ls` – list contents of a directory
  - `pwd` – print/show current working directory
  - `cd` – change current directory
  - `mkdir` – make a new directory
  - `rmdir` – remove/delete a directory
- Files
  - `cp` – copy a file to a new name and/or location
  - `mv` – move a file to a new name and/or location
  - `rm` – remove/delete a file (or directory)
  - `grep` – search content of a file for some text
- Editor
  - `nano` – a full screen interactive editor to edit text files
- Remote login
  - `ssh` - establishes a secure connection from your computer to a remote computer
- Transferring data between systems
  - `scp` - x
  - `rsync` - x
- Bash shell
  - `bash` - the shell program that
    - accepts user commands, runs programs, and shows output on the screen
    - can run commands from a script file, run programs, and sends output to another file
  - Environment variables - store values that `bash` and other programs can use during execution
  - `PATH` - variable containing the set of directories bash searches to run a program
  - `LD_LIBRARY_PATH` - variable contains set of directories a program uses to link its dynamic libraries
- Other
  - `which` - shows the directory from which a given program will be executed
  - `ldd` - displays the directories from which each dynamically linked library will be loaded



The `module` command

- **`module`** manages **`PATH`** and other environment variables with simple commands (see https://modules.sourceforge.net/)
- Each of the following SW is installed in a different directory
  - GNU compilers versions 5, 7, 9, 12, 14
  - Ansys, Fluent
  - Gaussian, Amber
  - …
- To run SW from a directory, need to include directory in **`PATH`** and **`LD_LIBRARY_PATH`** variables
  - Too much work
- For various software, the **`module`** command can be used to:
  - find,
  - list,
  - load, and
  - unload

  "modules" related to specific software and their versions



The **`module`** command

- By default, the following modules are loaded on Juno (as of July 2026)

`$ module list`

```
Currently Loaded Modules:
  1) autotools    3) gnu14/14.2.0    5) ucx/1.18.0          7) openmpi5/5.0.7
  2) prun/2.2     4) hwloc/2.12.0    6) libfabric/1.18.0    8) ohpc
```

- **`module`** sub-commands
  - **`module avail`**
  - **`module list`**
  - **`module load <module-name>`**
  - **`module unload <module-name>`**
  - **`module unload`**

Software available on Juno

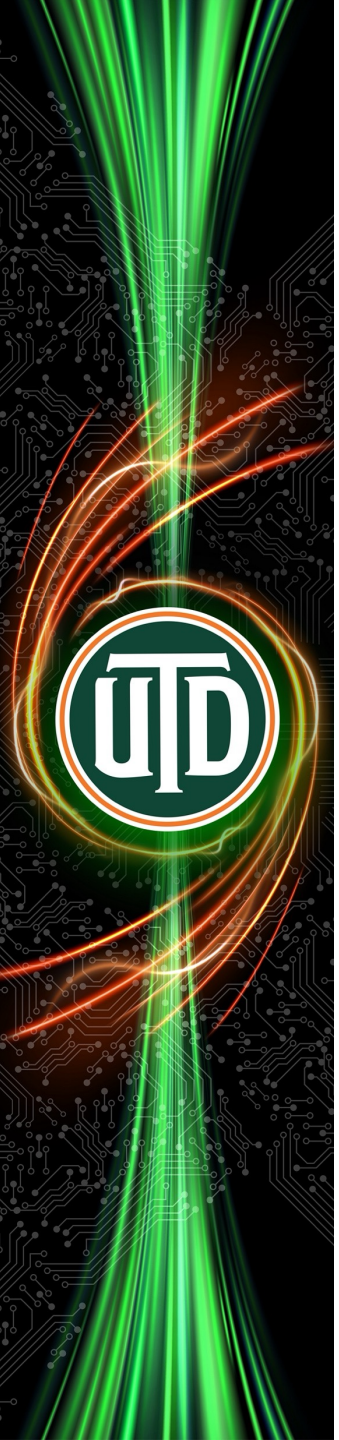
- Compilers and toolkits:
  - GNU, Intel
  - OpenMPI, MPICH
  - CUDA
  - AMD μProf, Valgrind
  - Launcher
  - Spack
  - Apptainer (container)
  - Miniconda
- Software and packages:
  - MATLAB
  - Gaussian
  - Ansys (Fluent, Lumerical,…)
  - GROMACS, AMBER, NAMD
  - R, Python
  - ORCA, QChem, VASP
  - Stata
  - Ollama

  and more…

Please use the **`module avail`** command for the full list of software

Software available on Juno

```
(base) [dal281726@juno-l-02 ~]$ module avail

------------------------------------------------------------------------ /opt/ohpc/pub/moduledeps/gnu14-openmpi5 -------------------------------------------------------------
   adios2/2.10.1    fftw/3.3.10     mumps/5.2.1                omb/7.5              phdf5/1.14.6       scalapack/2.2.2    slepc/3.18.0
   boost/1.88.0     hypre/2.33.0    netcdf-cxx/4.3.1           opencoarrays/2.10.2  pnetcdf/1.14.0     scalasca/2.6.2     superlu_dist/6.4.0
   dimemas/5.4.2    imb/2021.3      netcdf-fortran/4.6.2 (D)   otf2/3.1.1           ptscotch/7.0.7     scorep/9.0         tau/2.31.1
   extrae/3.8.3     mfem/4.4        netcdf/4.9.3               petsc/3.18.1         py3-mpi4py/3.1.5   sionlib/1.7.7      trilinos/13.4.0

------------------------------------------------------------------------ /opt/ohpc/pub/moduledeps/gnu14 ------------------------------------------------------------------
   R/4.5.0        gotcha/1.0.8        likwid/5.4.1       mpich/3.4.3-ucx (D)    openmpi4/4.1.6          plasma/24.8.7       superlu/7.0.0
   cubelib/4.9    gsl/2.8             metis/5.1.0        opari2/2.0.9           openmpi5/5.0.7   (L)    py3-numpy/1.26.4
   cubew/4.9      hdf5/1.14.6  (D)    mpich/3.4.3-ofi    openblas/0.3.29        pdtoolkit/3.25.1        scotch/7.0.7

------------------------------------------------------------------------ /opt/ohpc/pub/modulefiles -----------------------------------------------------------------------
   EasyBuild/5.0.0               autotools            (L)    gnu12/12.4.0                lichem/lichem                    papi/6.0.0
   abaqus/2023                   bcl/4.3.1                   gnu13/13.2.0                lua/5.4.7                        pmix/4.2.9
   abaqus/2026            (D)    castep/25.12                gnu14/14.2.0         (L)    matlab/r2024b                    prun/2.2                  (L)
   afni/25.3.03                  charliecloud/0.15           gnu5/5.5.0                  miniconda/24.11.1                python/3.11.11
   amber/24-beta                 cmake/4.0.0                 gnu8/8.5.0                  namd/3.0.1                       python/3.12.2             (D)
   amber/24               (D)    code-server/3.10.2          gnu9/9.3.0                  nccl-tests/cuda12.4              qchem/6.2.2
   amduprof/5.1.7                code-server/4.100.2         gnuplot/6.0.4               nccl/2.29.2-1-cuda12.4           qchem/6.3                 (D)
   ansys/2025R1                  code-server/4.108.2  (D)    gromacs/2024.5-plumed       netcdf-c/4.9.2-nvhpc             qe/7.3.1
   ansys2024R2/ansys2024R2       comsol/6.4                  gurobi/12.0.1               netcdf-fortran/4.6.1-nvhpc       rosetta/3.15
   ansys2024R2/AnsysEM           cuda-q/0.12.0               hdf5/1.13.2-nvhpc           nonlinloc/7.1.04                 siesta/5.2.1-parallel
   ansys2024R2/autodyn           cuda/11.7                   hpcc/1.5                    nvhpc/24.11                      siesta/5.4.2-wannier90    (D)
   ansys2024R2/fluent            cuda/12.4                   hwloc/2.12.0         (L)    nvshmem/3.5.19-cuda12.4          spack/0.23.1
   ansys2024R2/Workbench  (D)    cuda/12.6                   intel/2023.2.0              ohpc                      (L)    stata/19.5
   ansys2025R1/ansys2025R1       cuda/13.0            (D)    intel/2025.0         (D)    ollama/12                        szip/2.1.1-nvhpc
   ansys2025R1/AnsysEM           cyberworkbench/10.1.7       java/11                     ollama/15                        tcad-silvaco/243422
   ansys2025R1/autodyn           diagham/0.01                jobstats/1.0                ollama/21                 (D)    tinker/9
   ansys2025R1/fluent            filetools/1.0               jq/1.7.1                    openfoam/2512                    ucx/1.18.0                (L)
   ansys2025R1/Workbench  (D)    fsl/6.0.7.19                julia/1.11.3                openmpi-aocc/5.0.6               valgrind/3.24.0
   aocc/5.0                      fvcom/5.0.1                 kempnerpulse/0.4.1          orca/6.0.1                       vasp/6.4.2-upgrade
   apptainer/1.3.4               gaussian/16                 launcher/3.9                orca/6.1.1                (D)    vasp/6.4.2-wannier90-only (D)
   autodock_vina/1.2.7           gnu11/11.3.0                libfabric/1.18.0     (L)    os

  Where:
   D:  Default Module
   L:  Module is loaded

If the avail list is too long consider trying:

"module --default avail" or "ml -d av" to just list the default modules.
"module overview" or "ml ov" to display the number of modules for each name.

Use "module spider" to find all possible modules and extensions.
Use "module keyword key1 key2 ..." to search for all possible modules matching any of the "keys".
```



Logging into Juno and transferring data

SSH (command line)

Open OnDemand (web based)

Transferring data into and out of Juno

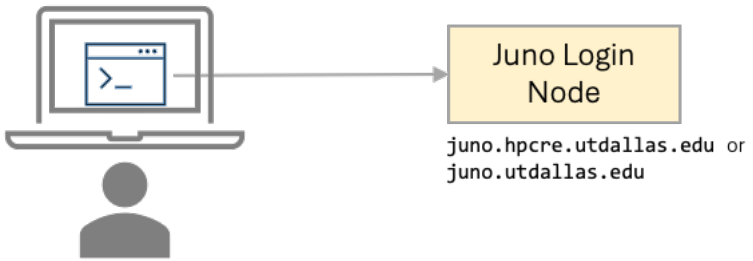
Logging into Juno: ssh

- Your computer must be connected to the the UTD campus network via
  - Wired Ethernet connection on campus, or
  - WiFi connection to CometNet, or
  - A VPN connection to campus (see https://atlas.utdallas.edu/TDClient/30/Portal/Requests/ServiceDet?ID=167)
- Start a terminal program on your computer
- To login to Juno, run

  `ssh <your-NetID>@juno.utdallas.edu`
  e.g.,
  `ssh ksw240000@juno.utdallas.edu`
- **`ssh`** establishes a secure remote connection from your computer to the Juno login node

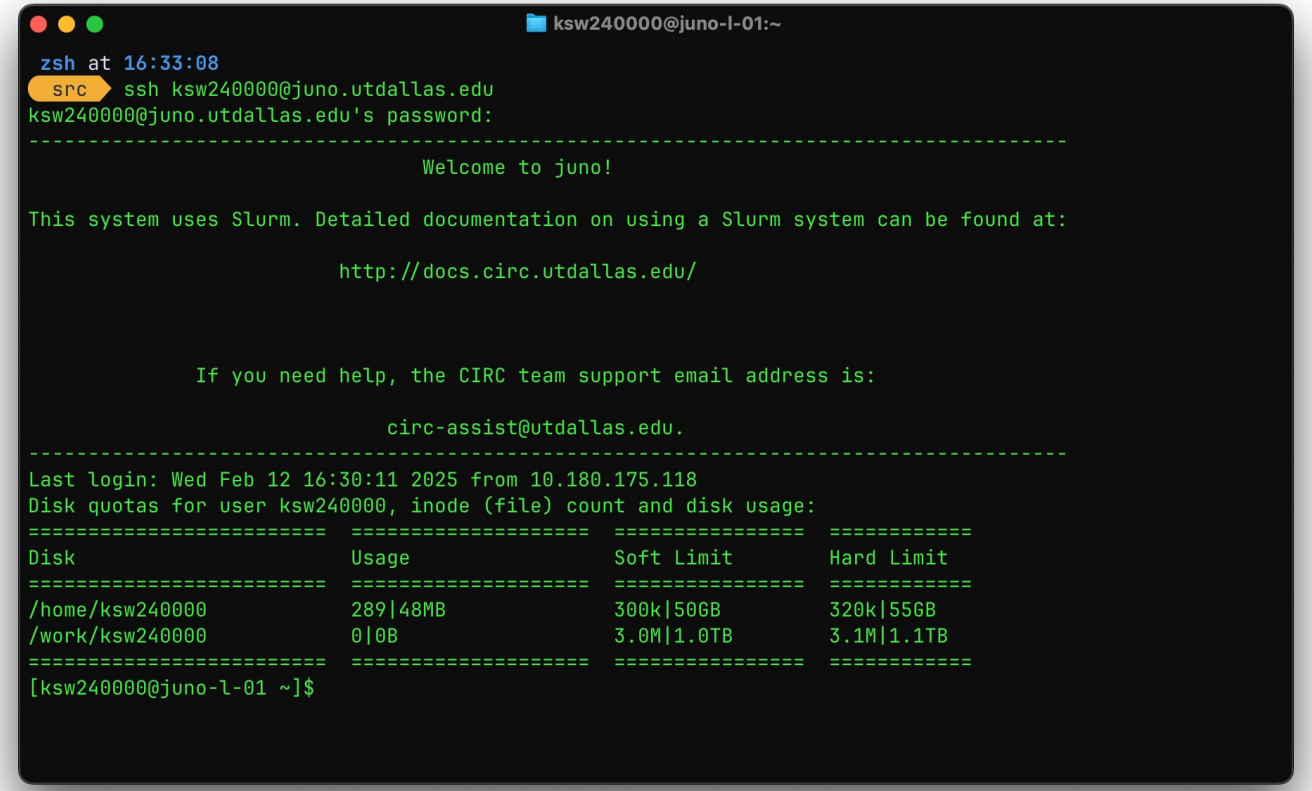
Recommended terminal programs

- Windows: Windows Terminal, MobaXTerm, or putty
- Mac: Terminal (pre-installed) or Ghostty
- Linux: terminal (pre-installed) or xterm (pre-installed)



Command line interface (CLI)

Logging into Juno: ssh



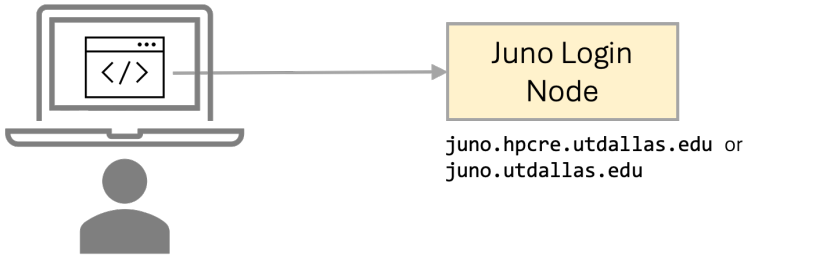
- You will be asked to
  - Verify that you want to connect to Juno (once only, not show on left)
  - Enter your password (associated with your NetID)
- You will see
  - A welcome message
  - Current disk quotas
  - Juno command prompt

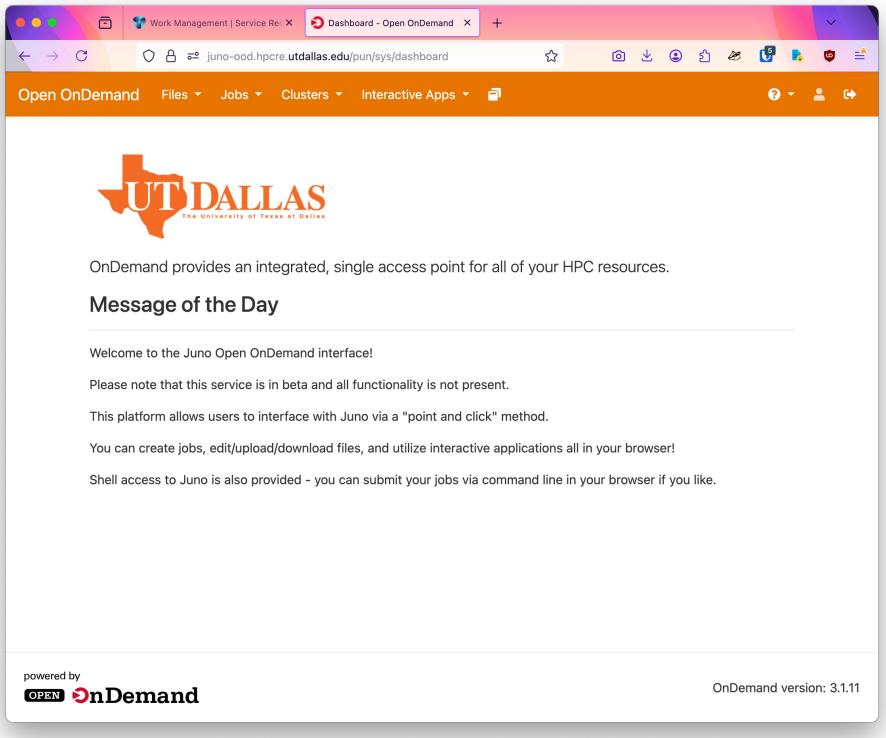
```
[ksw240000@juno-l-01 ~] $
```

- You are ready to use Juno

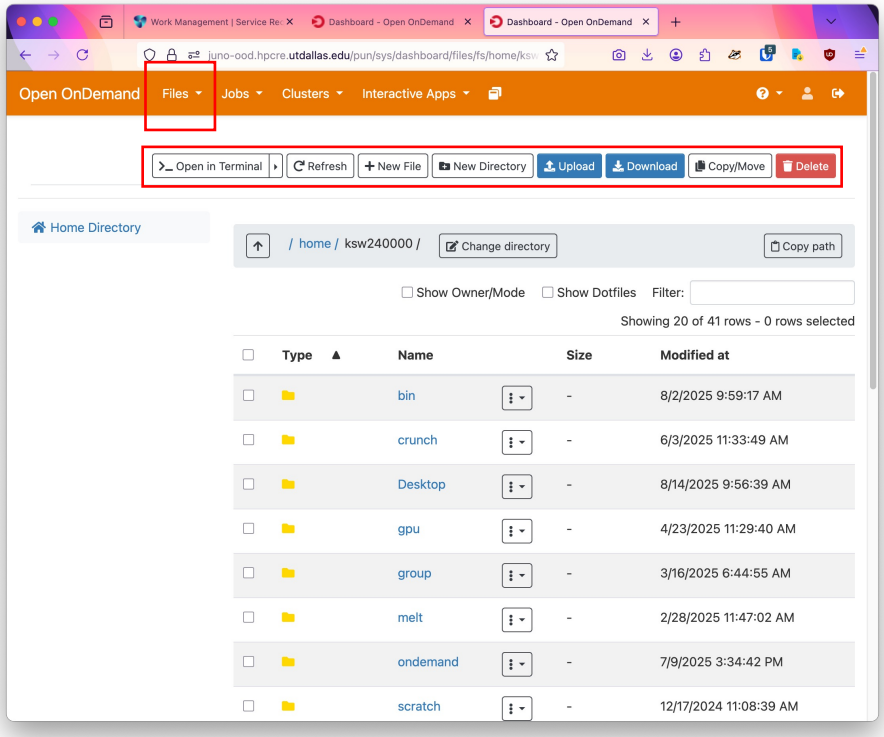
Logging into Juno: Open OnDemand

- Juno Open OnDemand portal lets users transfer files and run select programs
  - https://juno-ood.hpcre.utdallas.edu/
- Login using NetID and password





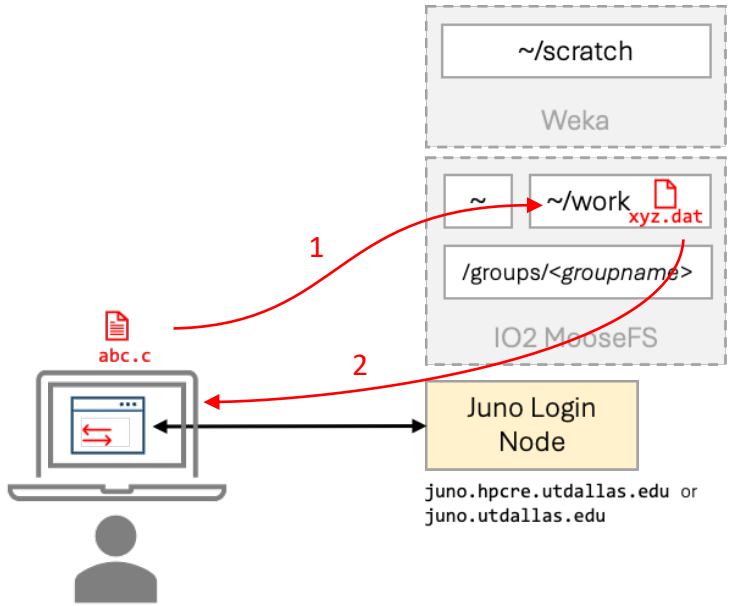
Open OnDemand files access



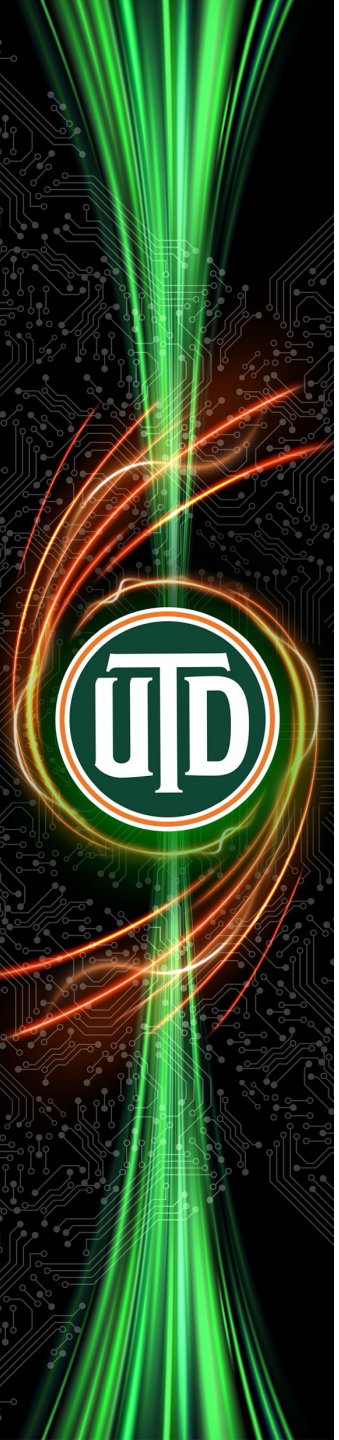
- You can browse files
- Upload files (up to 10 GB)
- Download files (up to 10 GB)

Transferring data

- You can transfer data between your computer and Juno using a data transfer program
- Examples of data transfer programs
  - **`scp`**, **`rsync`**
  - Filezilla




```
1 $ scp abc.c ksw240000@juno.utdallas.edu:~/work
2 $ scp ksw240000@juno.utdallas.edu:/work/xyz.dat xyz.dat
```

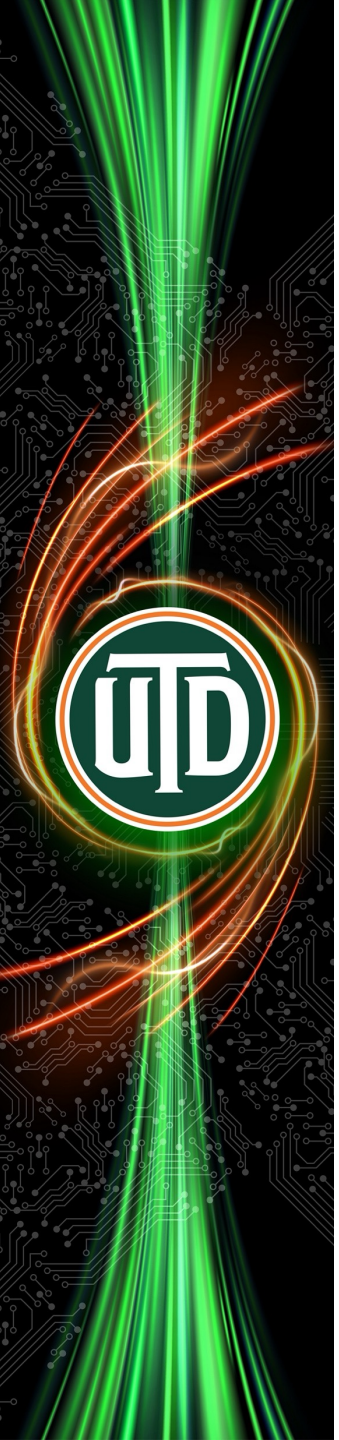


rsync

- **`rsync`** can be useful to run on slow and unreliable connections.
- If your download aborts in the middle of a large file, **`rsync`** will be able to continue from where it left off when invoked again.

```
rsync -P <local-file> <remote-user>@<remote-system>:<remote-directory>
```

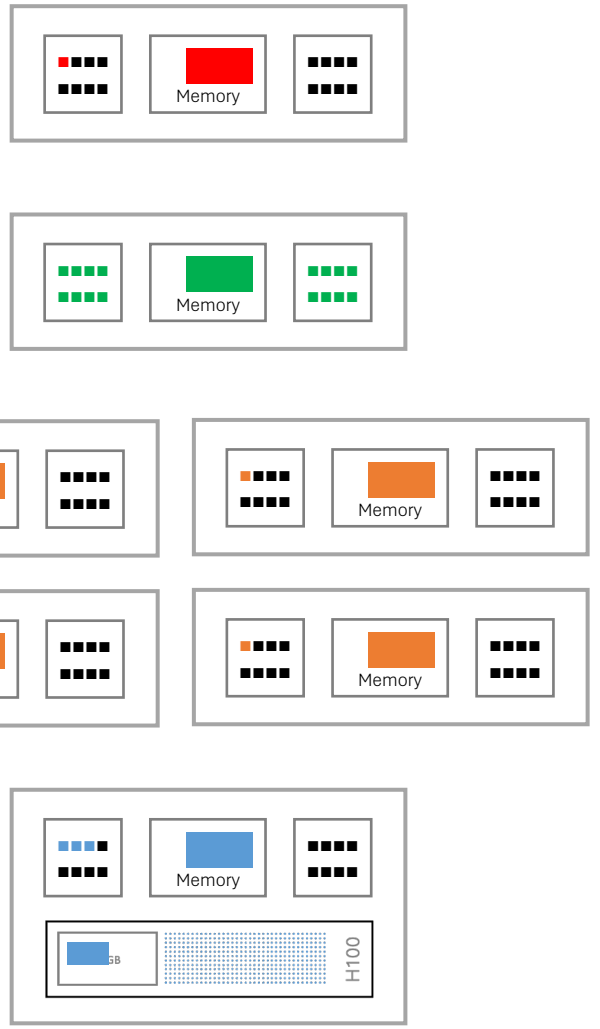
- The **`-P`** option preserves partially downloaded files and also shows progress.



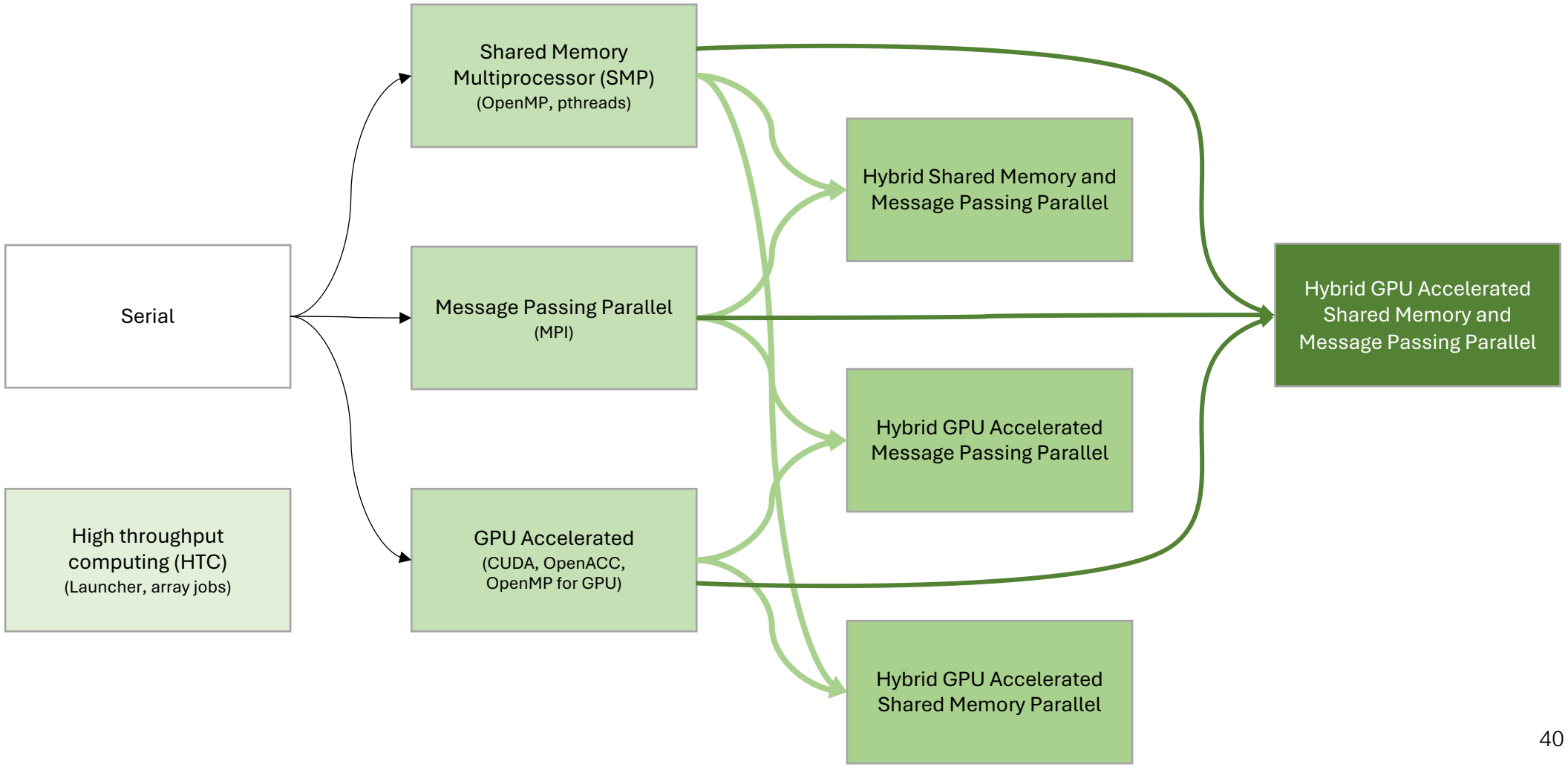
Achieving high performance using parallel processing

Achieving high performance on an HPC cluster

- Programs normally perform their work on just one core
  - This is called serial programs
- Achieving high performance requires the program to split their work and perform it simultaneously on multiple cores
  - This is called parallel programs
- On an HPC cluster, parallel execution can happen in 3 ways
  - Use multiple cores in single node (shared memory multiprocessor (SMP) or multithreaded parallel)
    - Using OpenMP or pthreads
  - Use cores in multiple nodes (message passing or distributed memory parallel)
    - Using MPI
  - Use the tiny "processors" in GPUs (GPU accelerated)
    - Using CUDA or OpenMP for GPUs



Models of parallel processing on HPC clusters

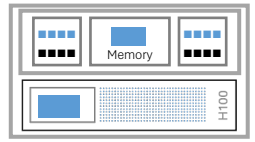


Most common types of parallel programs

Serial

Shared Memory Multiprocessor (SMP)
(OpenMP, pthreads)

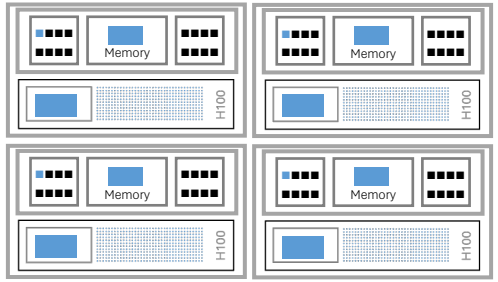
Hybrid GPU Accelerated Shared Memory Parallel



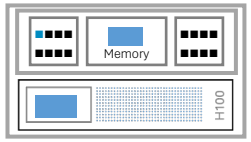
High throughput computing (HTC)
(Launcher, array jobs)

Message Passing Parallel
(MPI)

Hybrid GPU Accelerated Message Passing Parallel

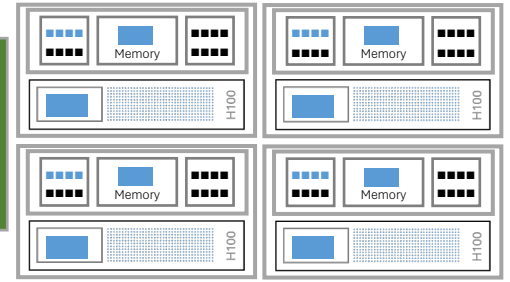


GPU Accelerated
(CUDA, OpenACC, OpenMP for GPU)



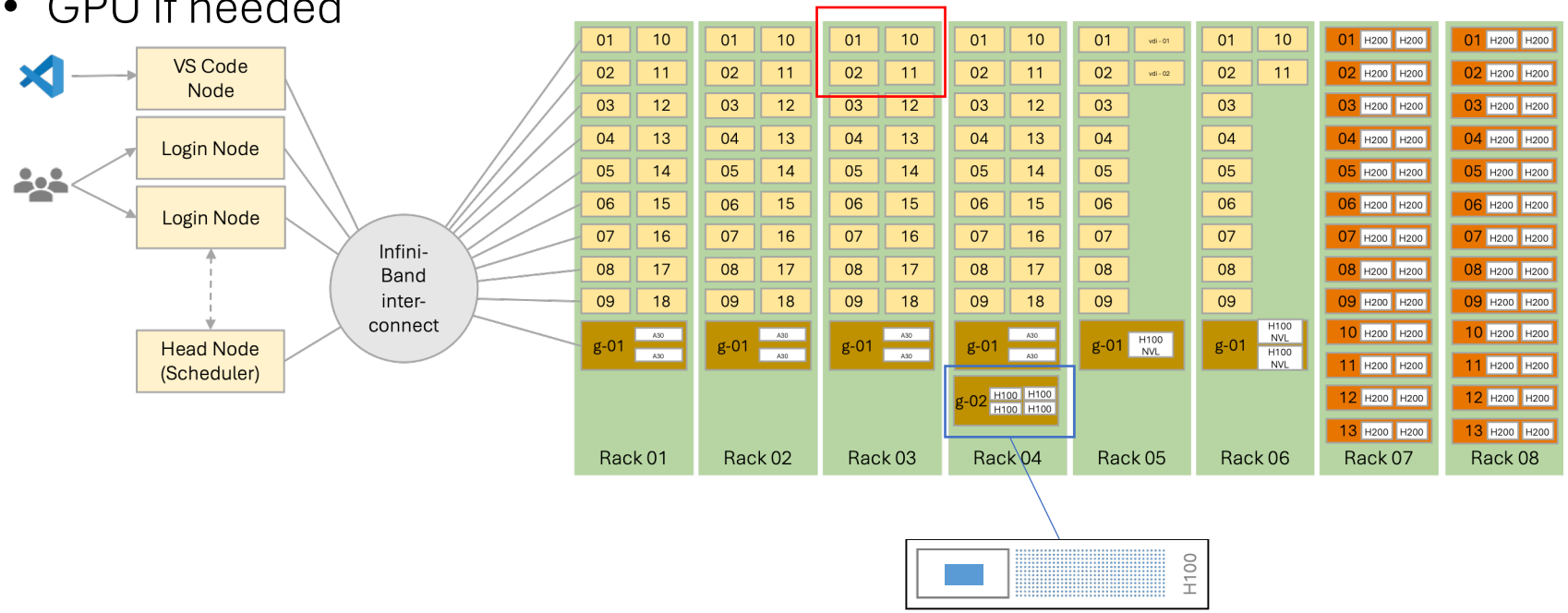
Hybrid Shared Memory and Message Passing Parallel

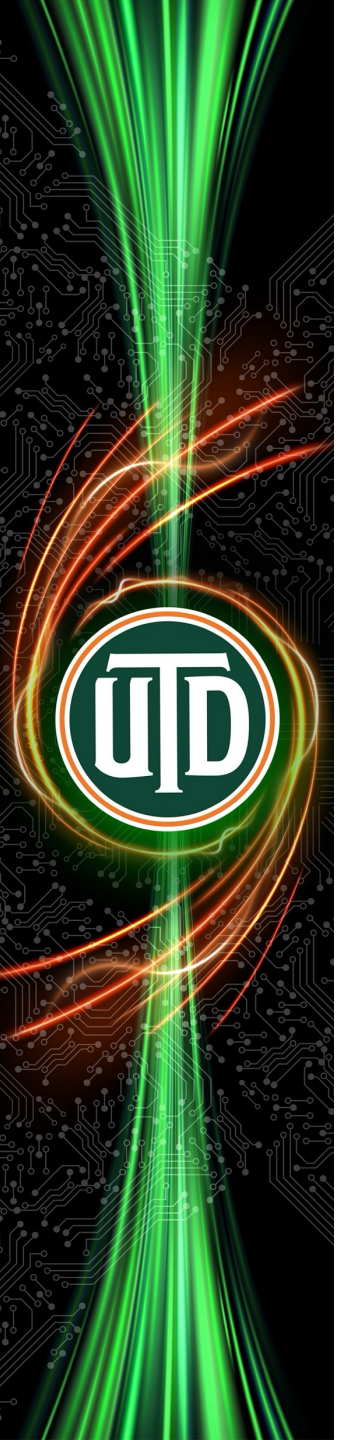
Hybrid GPU Accelerated Shared Memory and Message Passing Parallel



What resources does your program need?

- Depending on what parallelism modes you're using and how you map your problem to available resources, every program requests:
  - Number of nodes: number of compute nodes
  - CPU cores: number of processes you want to run in parallel
  - Random access memory (RAM)
  - GPU if needed

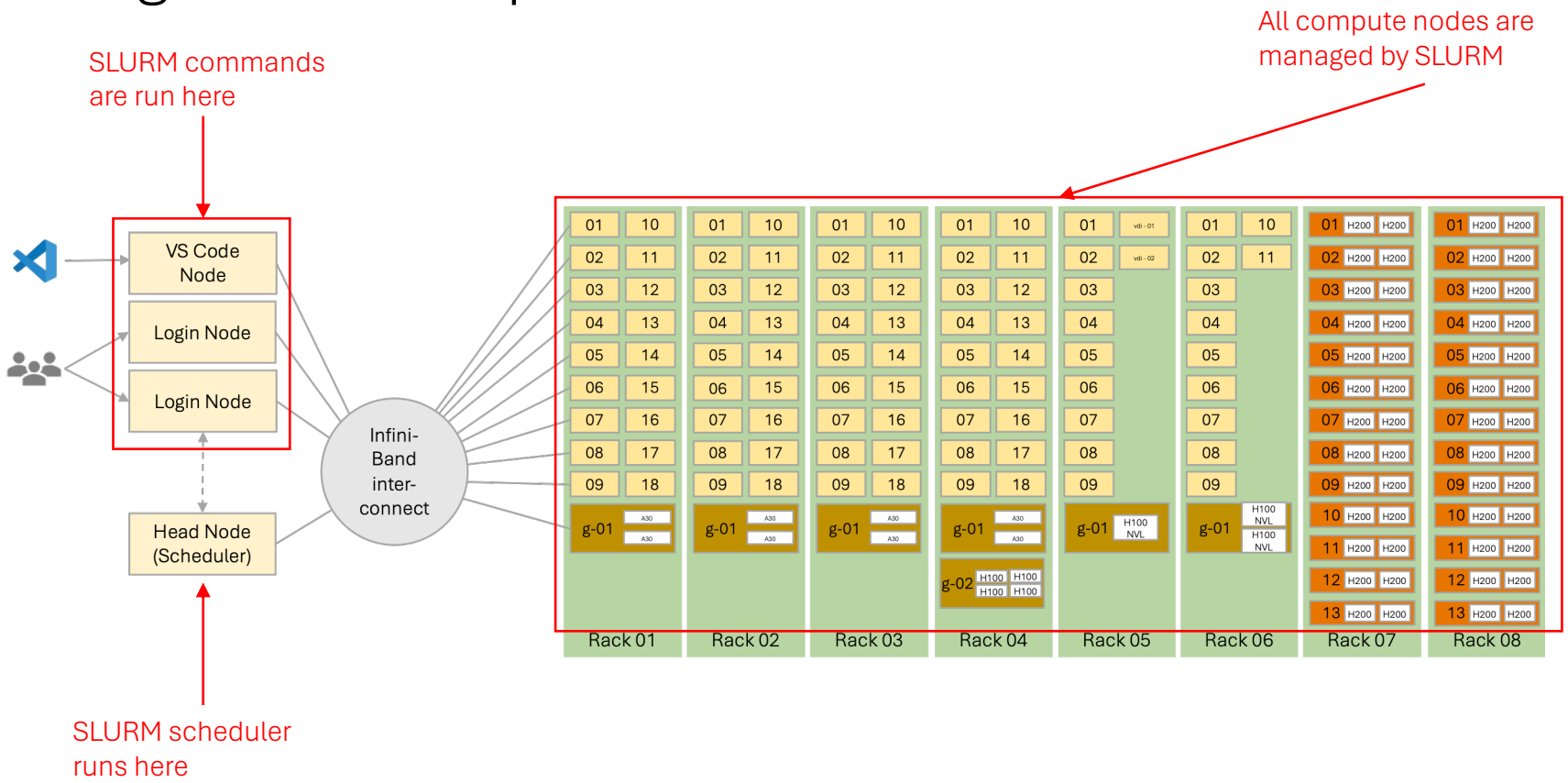




Introduction to the SLURM job scheduler

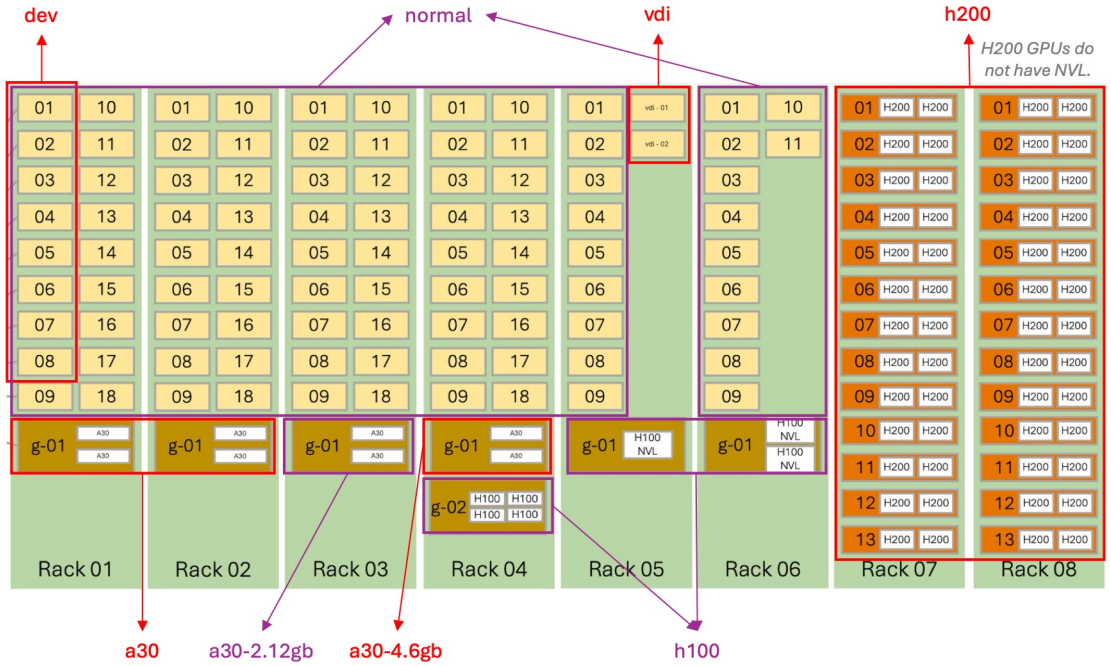
What is the SLURM job scheduler

- SLURM is a job scheduler that manages the programs running on the compute nodes.



SLURM partitions (often also called queues)

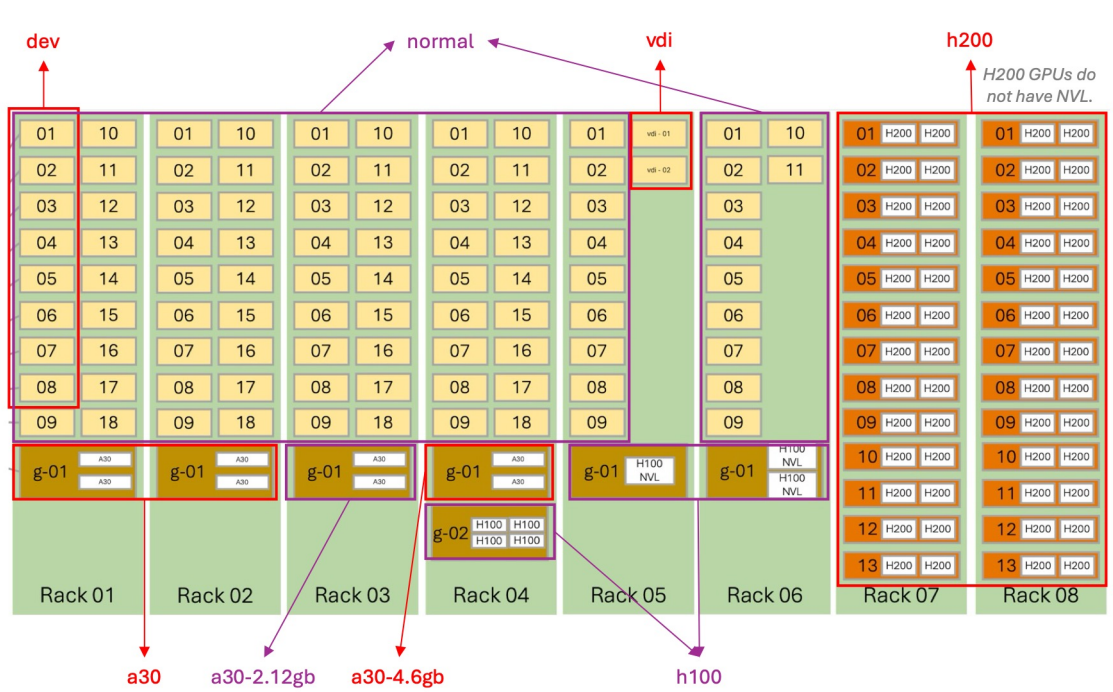
- Juno nodes are grouped into partitions. Each partition may have the following resources:
  - Nodes
  - CPUs (cores)
  - Memory
  - GPUs (full physical or fractional “virtual”)
  - Time limit (how long a job can run)
- You submit your jobs to the partition that has the resources matching your job’s needs
- SLURM runs the job when the resources become available
- **`sinfo`** command shows the currently configured SLURM partitions



SLURM partitions (sometimes called queues)

```
[dal281726@juno-l-01 ~]$ sinfo
```

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
normal*	up	2-00:00:00	9	mix-	c-01-[07-08],c-02-14,c-03-07,...
normal*	up	2-00:00:00	1	drain	c-01-03
normal*	up	2-00:00:00	5	mix	c-01-[05,15],c-02-18,c-04-18,c-05-06
normal*	up	2-00:00:00	75	alloc	c-01-[04,06,09-14,16-18],...
a30	up	2-00:00:00	2	mix-	g-01-01,g-02-01
a30-2.12gb	up	2-00:00:00	1	mix-	g-03-01
a30-4.6gb	up	2-00:00:00	1	drain	g-04-01
dev	up	2:00:00	2	mix-	c-01-[07-08]
dev	up	2:00:00	1	mix	c-01-05
dev	up	2:00:00	3	alloc	c-01-[04,06,09]
dev	up	2:00:00	2	idle	c-01-[01-02]
h100	up	2-00:00:00	3	mix-	g-04-02,g-05-01,g-06-01
vdi	up	8:00:00	2	idle	v-05-[01-02]



Juno SLURM configuration

Partition name	Time limit	Nodes	Max nodes/job	Cores/ node	Memory/ node◆	GPUs/ node	VRAM/ GPU	Best used for
dev	2 hours	8*	4	64	384 GB	-	-	Code development, short jobs, benchmarking
normal	2 days	92*	8	64	384 GB	-	-	Long jobs, big jobs, production (main) compute workloads
h200‡	2 days	26	8	64	384 GB	**2** H200 physical (141 GB)	141 GB	Large, long jobs requiring high GPU resources
h100‡	2 days	1	1	64	512 GB	**4** H100 physical	80 GB	Large, long jobs requiring high GPU resources
		1	1	64	512 GB	**2** H100 physical	94 GB	
		1	1	64	512 GB	**1** H100 physical	94 GB	
a30‡	2 days	2	2	128	1,024 GB	**2** A30 physical	24 GB	Large, long jobs requiring medium GPU resources
a30-2.12gb‡	2 days	1	1	128	1,024 GB	**4** half-A30 virtual	12 GB	Large, long jobs requiring small GPU resources (*e.g.* Launcher + GPU)
a30-4.6gb‡	2 days	1	1	128	1,024 GB	**8** quarter-A30 virtual	6 GB	
vdi	8 hours	2	1	64	384 GB	**-**		Used by GUI-interactive workload

*__dev__ partition shares nodes with the __normal__ partition.

◆ *If a job does not specify its memory needs, then 64GB default memory limit will be applied.*

‡ ***--gres=gpu**:<n> sbatch option is required to access GPUs, where <n> is the number of GPUs requested.*

Job based limits

- Max nodes per job is 8

User based limits

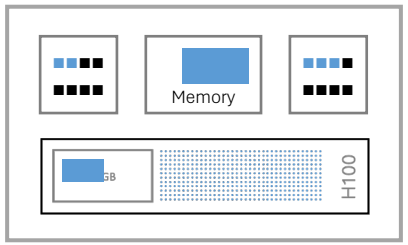
- Max running jobs per user is 4
- Max submitted jobs per user is 100

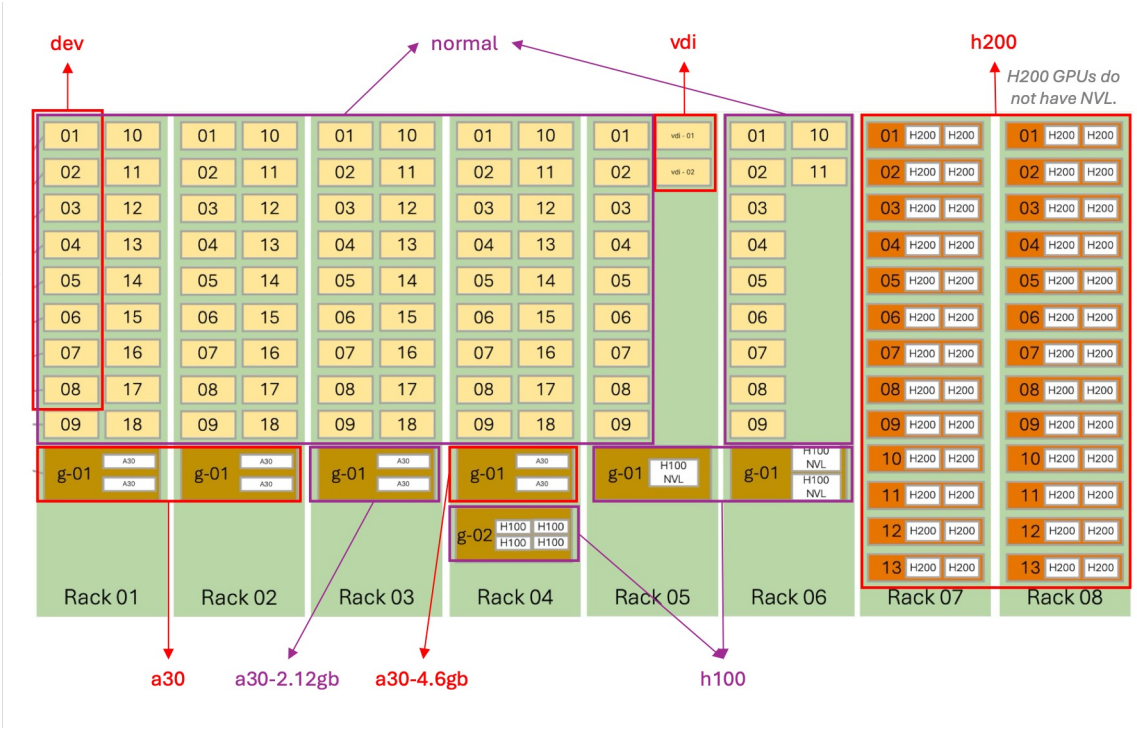
These limits can be relaxed for specific projects for limited time for codes that show efficient speedup at higher number of cores. Please contact us to temporarily raise your limit. We wish to support very large jobs on Juno but need to ensure that the jobs are using resources efficiently.

Choosing the right partition for your job

- Determine your job's resource needs?
  - How many nodes?
  - How many cores (CPUs) per node?
  - How much memory per node?
  - Do you need GPUs? If so, which kind?
  - How long does your program need to run?
- If your job needs 1 node, 6 CPU cores per node, 30GB memory and 3.5 hours to run, then your batch job file will have the following options specified:

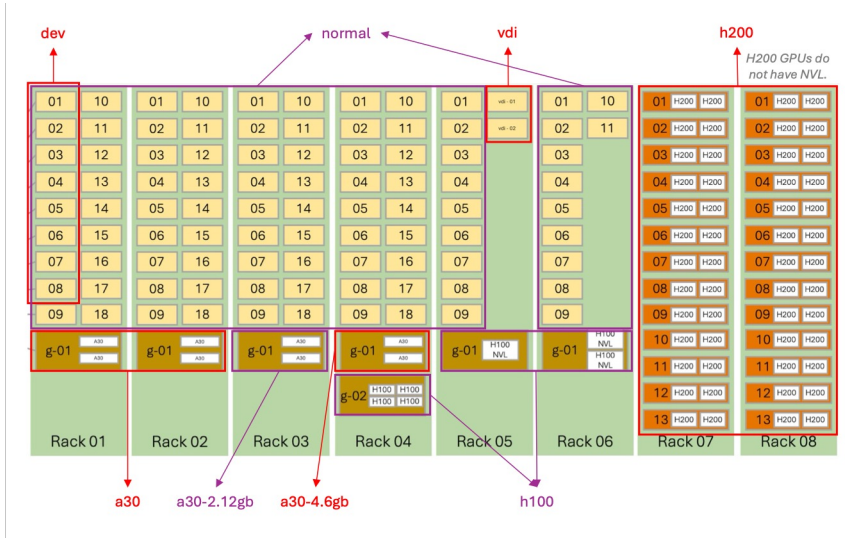
```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=6
#SBATCH --mem=30G
#SBATCH --time=3:30:00
#SBATCH --gres=gpu:1
#SBATCH --partition=h100
```



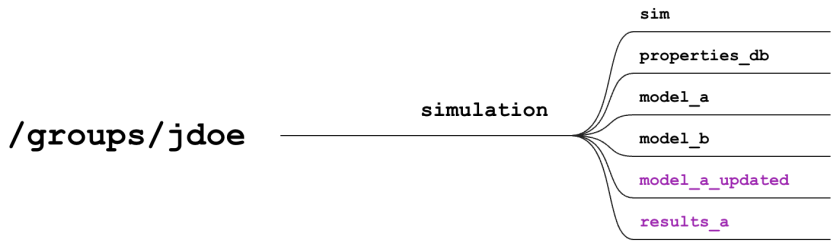


Choosing the right partition examples

- Machine learning / Deep learning training with multiple GPUs
  - Light model: **a30** partitions
  - Big model: **h100**, **h200** partitions
- OpenMP programs
  - **normal** partition, **a30** for more intensive RAM usage
- MPI programs
  - **normal** partition (no GPU needed), **a30** or **h200** (GPU needed)
- Compiling, debugging, profiling
  - **dev** partition



Scratch space

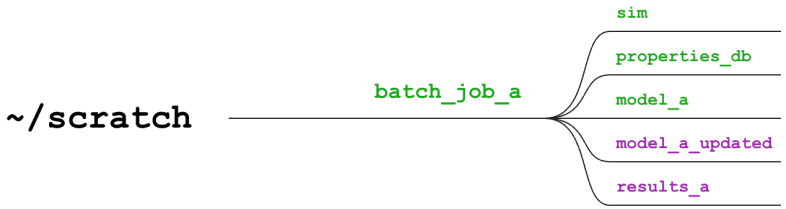


Scratch

~/scratch

30TB (soft limit)

Private to user, never backed up, purged regularly



Use for I/O in batch jobs
Scratch is up to 10x faster at large I/O than **~**, **~/work** or **/groups**

Data movement to Scratch occurs in the following sequence:

1. **Copy** data **in** at the start
2. **Read** from and **write** to Scratch **during** the job
3. **Copy** data **out** at the end
4. **Clean up** before leaving

```
1  $ cd ~/scratch
   $ mkdir batch_job_a
   $ cd batch_job_a

   $ cp /groups/jdoe/simulation/sim .
   $ cp /groups/jdoe/simulation/properties_db .
   $ cp /groups/jdoe/simulation/model_a .

2  $ ./sim model_a results_a

3  $ cp model_a_updated /groups/jdoe/simulation
   $ cp results_a /groups/jdoe/simulation

4  $ cd ..
   $ rm -rf batch_job_a
```

Scratch space

Scratch

~/scratch

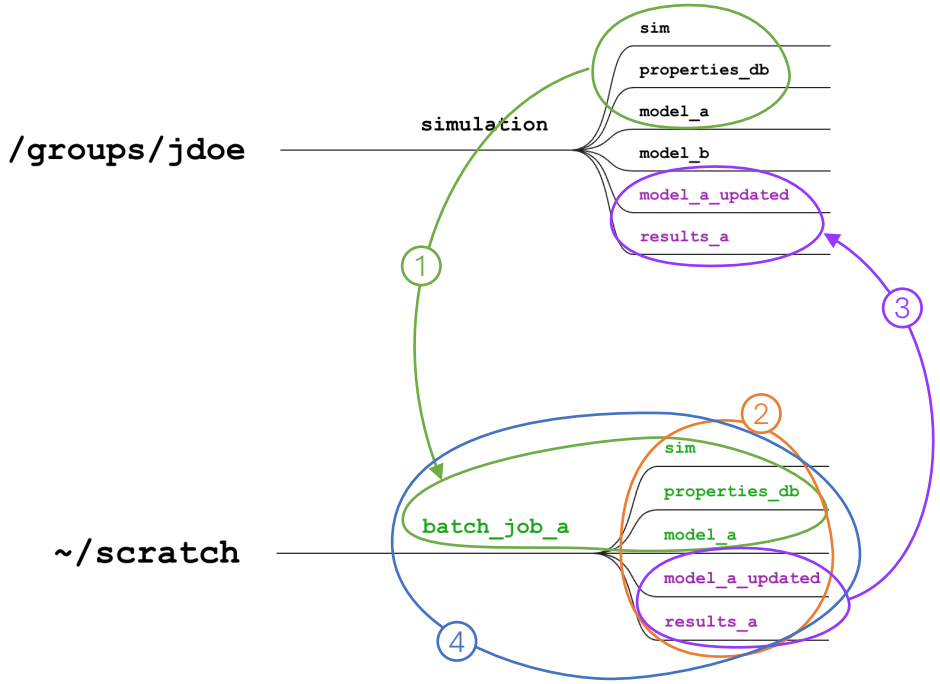
30TB (soft limit)

Private to user, never backed up, purged regularly

Use for I/O in batch jobs
Scratch is up to 10x faster at large I/O than **~**, **~/work** or **/groups**

Data movement to Scratch occurs in the following sequence:

1. **Copy** data **in** at the start
2. **Read** from and **write** to Scratch **during** the job
3. **Copy** data **out** at the end
4. **Clean up** before leaving



1
```
$ cd ~/scratch
$ mkdir batch_job_a
$ cd batch_job_a

$cp /groups/jdoe/simulation/sim .
$cp /groups/jdoe/simulation/properties_db .
$cp /groups/jdoe/simulation/model_a .
```

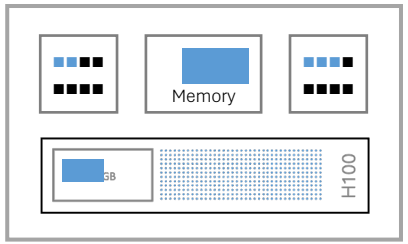
2
```
$ ./sim model_a results_a
```

3
```
$ cp model_a_updated /groups/jdoe/simulation
$ cp results_a /groups/jdoe/simulation
```

4
```
$ cd ..
$ rm -rf batch_job_a
```

What's inside a SLURM job.sh?

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=6
#SBATCH --mem=30G
#SBATCH --time=3:30:00
#SBATCH --gres=gpu:1
#SBATCH --partition=h100
#SBATCH --job-name=sim_model_a
#SBATCH --output=sim_model_a_output.txt

# List and load the required modules
module list
module load cuda/12.4
```

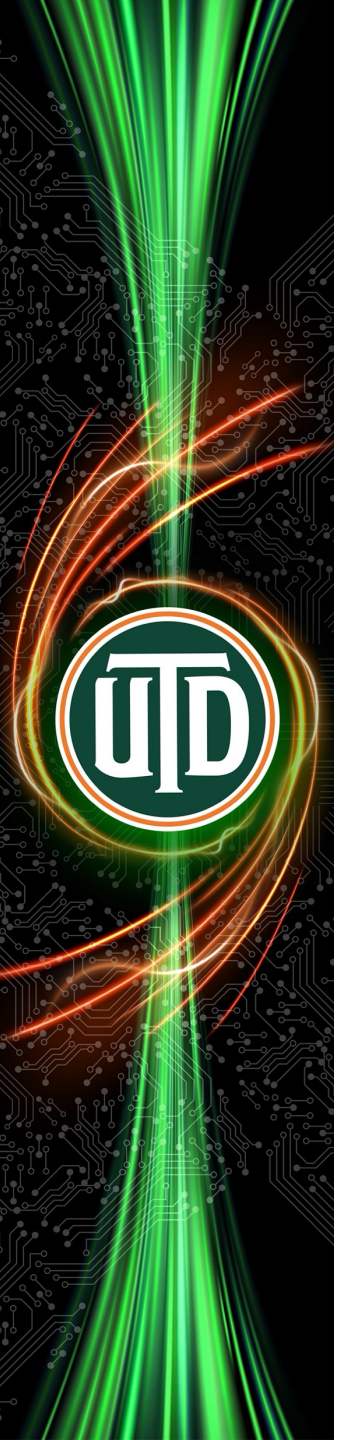



```
# Batch files reside in the following directory
SRC=/groups/hpcre/orientation/simulation/

# Copy data IN at the start
cd ~/scratch
mkdir batch_job_a
cd batch_job_a

cp $SRC/sim .
cp $SRC/properties_db .
cp $SRC/model_a .

# Read from and write to Scratch during the job
# Run your job's commands
./sim model_a results_a

# Copy data OUT at the end
cp model_a_updated $SRC
cp results_a $SRC

# Clean up before leaving
cd ..
rm -rf batch_job_a
```

Submitting and managing SLURM jobs

- **sbatch <batchfile>**
  - Submit a SLURM batch job, requires a batch job script filename
- **squeue --me**
  - List the queued and running batch jobs, use –u option if you only want to see your jobs
- **scancel <jobid>**
  - Cancel a submitted job, requires one or more job IDs

```
(base) [ksw240000@juno-l-02 simulation]$ pwd
/groups/hpcre/orientation/simulation
(base) [ksw240000@juno-l-02 simulation]$ ls
job.sh  model_a  model_b  properties_db  sim
(base) [ksw240000@juno-l-02 simulation]$ sbatch job.sh
Submitted batch job 2540797
(base) [ksw240000@juno-l-02 simulation]$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
           2540797 h100      sim_mode ksw24000  R       0:04      1 g-01-04
(base) [ksw240000@juno-l-02 simulation]$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
           2540797 h100      sim_mode ksw24000  R       0:17      1 g-01-04
(base) [ksw240000@juno-l-02 simulation]$ squeue --me
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
(base) [ksw240000@juno-l-02 simulation]$ ls
job.sh  model_a  model_a_updated  model_b  properties_db  results_a  sim  sim_model_a_output.txt
```

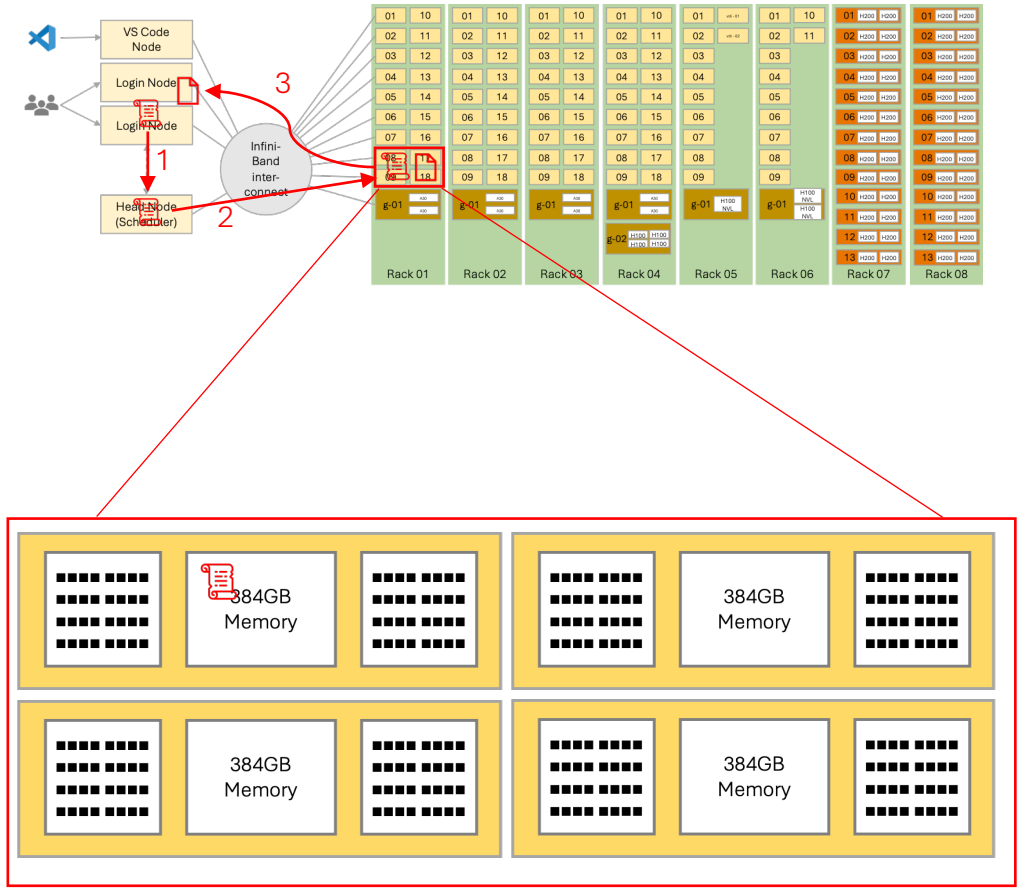


Job output file

```
(base) [ksw240000@juno-l-02 simulation]$ cat sim_model_a_output.txt

Currently Loaded Modules:
  1) autotools   3) gnu12/12.4.0   5) ucx/1.15.0          7) openmpi4/4.1.6
  2) prun/2.2    4) hwloc/2.7.2    6) libfabric/1.19.0    8) ohpc



Running simulation...
Input file: model_a
Output file: results_a
Reading model_a...
Reading properties_db...
Starting simulation
Writing results_a...
Saving restart file model_a_updated...
Completing simulation
```

How does your batch job run?

1. **`sbatch job.sh`**
2. The scheduler allocates 4 nodes and starts the **`job.sh`** script on one of them
3. At end of the job, the output file is returned to the directory where the job was submitted (from the login node)



Running interactive session using SLURM

- Two steps
  1. Use **`salloc`** to acquire resources from SLURM
  2. Use **`srun`** to start your interactive session on the allocated resources/nodes

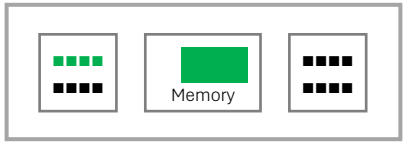
- **`salloc`** uses the same options as **`sbatch`** (nodes, tasks, time, GPU, etc.)

  `$ salloc -p normal -N 1 -n 1 -c 4 --mem=2GB`

- **`srun`** is used to start a Bash shell interactively on the resource allocation

  `$ srun --pty bash`
  - Your prompt will change to show the node name (*e.g.* c-04-18)

```
(base) [dal281726@juno-l-02 ~]$ salloc -N 1 -n 1 -c 4 --mem=2GB
salloc: Granted job allocation 87975
Disk quotas for user dal281726, inode (file) count and disk usage:
==========================  ====================  ================  ==========
Disk                        Usage                 Soft Limit        Hard Limit
==========================  ====================  ================  ==========
/home/dal281726             116k|33GB             300k|50GB         320k|55GB
/work/dal281726             17k|589GB             3.0M|1.0TB        3.1M|1.1TB
==========================  ====================  ================  ==========
(base) [dal281726@juno-l-02 ~]$ srun --pty bash
Disk quotas for user dal281726, inode (file) count and disk usage:
==========================  ====================  ================  ==========
Disk                        Usage                 Soft Limit        Hard Limit
==========================  ====================  ================  ==========
/home/dal281726             116k|33GB             300k|50GB         320k|55GB
/work/dal281726             17k|589GB             3.0M|1.0TB        3.1M|1.1TB
==========================  ====================  ================  ==========
(base) [dal281726@c-04-18 ~]$
```



Job scheduling algorithm

- To best serve our customers needs, we configure the scheduler optimize the following in a balanced way
  - Minimize the wait time of a user's jobs.
    - I.e., run the first job a user as soon as possible.
  - Maximize the number of users with running jobs.
    - I.e., run as many jobs from as many users concurrently as efficiently as possible.
  - Support large and long running computing jobs.
    - These represent the kind of computation that can only run on HPC systems.

The SLURM job scheduler computes the priority of each queued job and selects the job with highest priority to run next on the system.

The weight is calculated as follows.

Job's priority =

100,000 x (Fairshare of user) +
1,000 x (Job age factor) +
1,000 x (Job size factor)

- Each factor, its value and weight are show in the following table.

Factor	Weight	Value of factor	Note
Fairshare	100,000	0 to 1	See below for full explanation
Age of job	1,000	0 to 1	0 for new jobs and 1 for jobs 14 days or older
Size of job	1,000	0 to 1	Near 0 for small jobs, near 1 for large jobs (Size determined by CPUs, memory, time, and GPU resources of a job.)

Fairshare value

- In this example all numbers are selected for illustration (i.e., not real)
- All users in the same research group start with share of 1.0
  - Using the system decreases share
  - Not using the system gradually increases share (over time)
  - Share fully restored in 2 weeks
  - Share reset to 1.0 at the start of the month

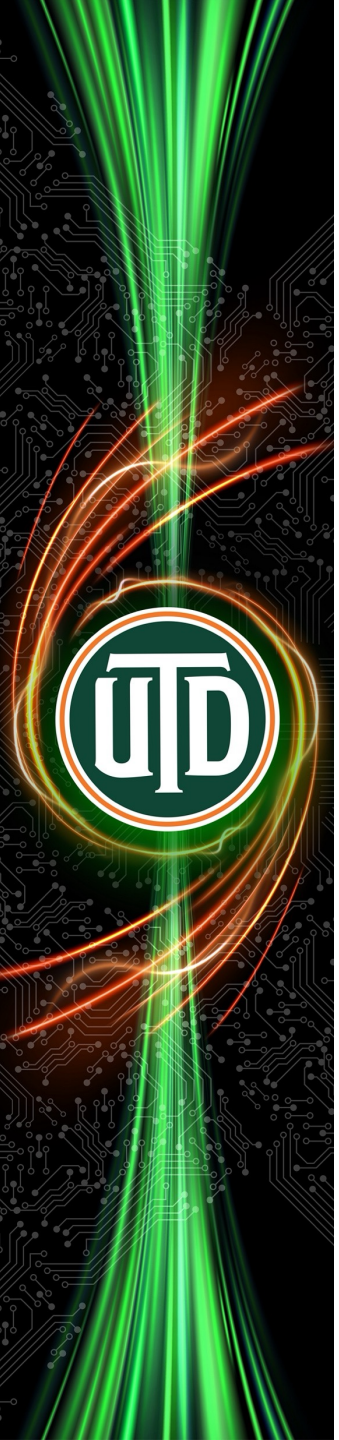
Event	User A Share	User B Share
Initially	1.0	1.0

How to monitor your share value?

- **`sshare`** tells you your, or every user's share value
- E.g.
- **`sshare`** (your share)
- **`sshare -a`** (all users' shares)

ksw240000@juno-l-01:~

```
juno(juno-l-01) ksw240000 ~ $  sshare -a
Account                    User  RawShares  NormShares    RawUsage  EffectvUsage  FairShare
-------------------- ---------- ---------- ----------- ----------- ------------- ----------
root                                          0.000000  2655590654      1.000000
 root                      root          1    0.333333           0      0.000000   1.000000
 hpcre                                   1    0.333333    26776917      0.010083
  hpcre                    amir          1    0.045455           0      0.000000   0.997297
  hpcre              axm240026          1    0.045455        6382      0.000238   0.956757
  hpcre              axv230014          1    0.045455        6507      0.000243   0.954054
  hpcre              brd150330          1    0.045455          20      0.000001   0.962162
  hpcre              cgd180001          1    0.045455          12      0.000000   0.964865
  hpcre              dal676969          1    0.045455           0      0.000000   0.997297
  hpcre              dal946607          1    0.045455      940376      0.035119   0.943243
  hpcre              exa230016          1    0.045455           9      0.000000   0.967568
  hpcre              kaa180001          1    0.045455    25054910      0.935691   0.940541
  hpcre                  kevin          1    0.045455           0      0.000000   0.997297
  hpcre              ksw240000          1    0.045455      156986      0.005863   0.948649
  hpcre              nxk230072          1    0.045455           0      0.000000   0.997297
  hpcre              rgr071000          1    0.045455           0      0.000000   0.997297
  hpcre                   root          1    0.045455         859      0.000032   0.959459
```



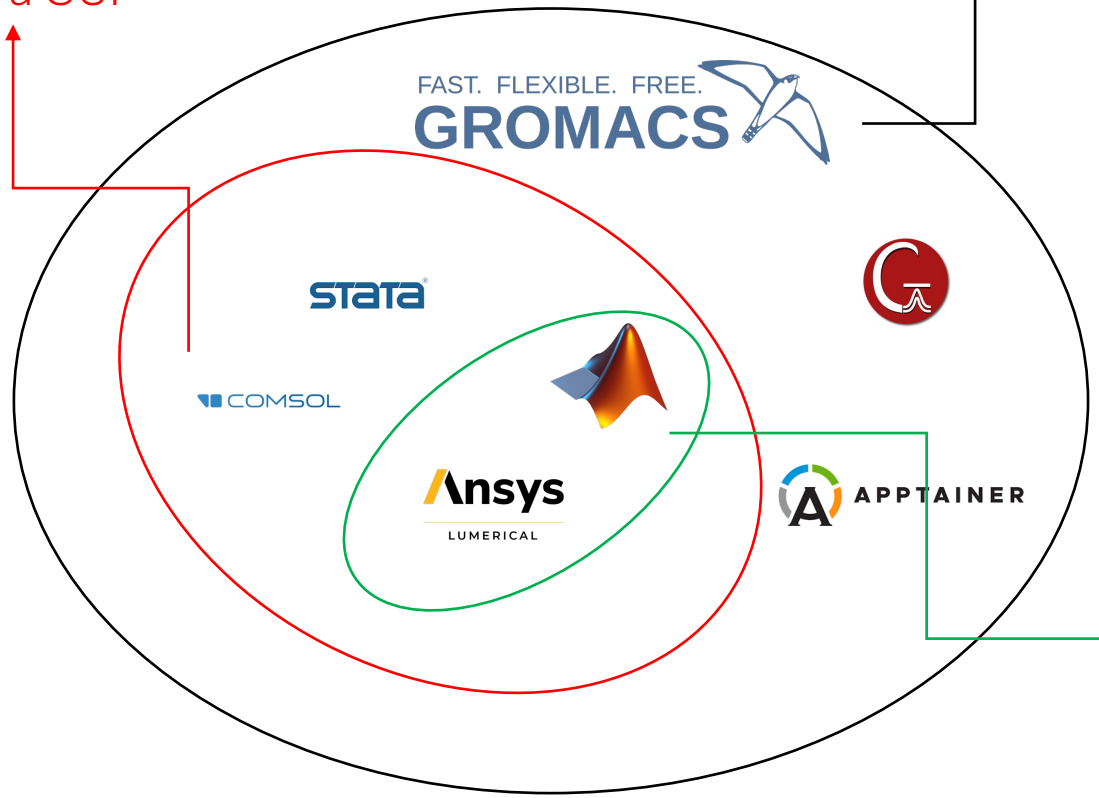
Other ways to run programs

Command lines and GUI

Ways to run program on a HPC cluster

A subset of programs offer a GUI

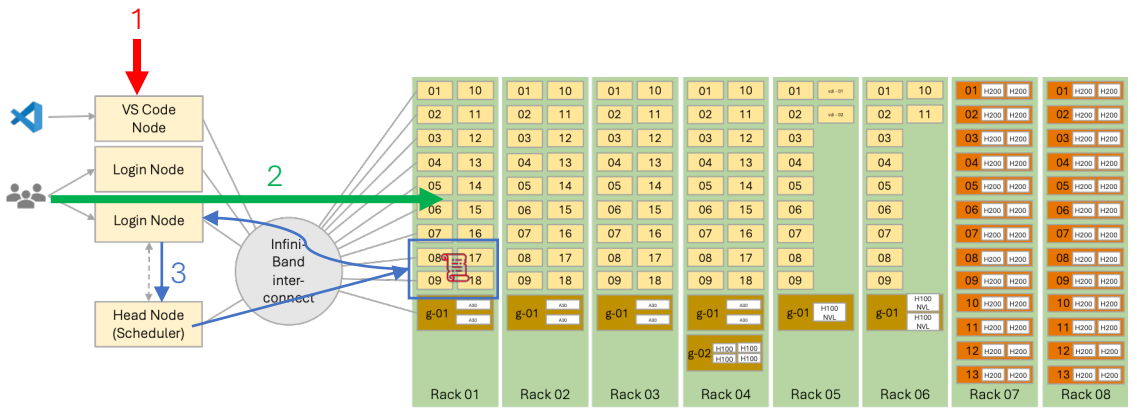
All programs are command line-based



A subset of programs offer offloading from a local workstation to a HPC cluster via **`ssh`**

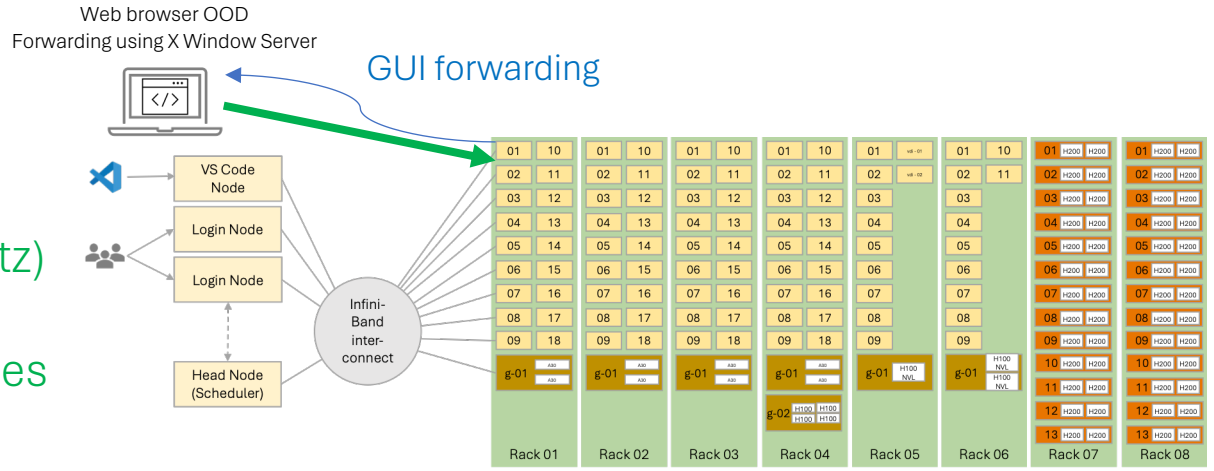
Command line interface (CLI)

1. Interactively on login node from the command line **(not recommended)**
2. Interactively on compute nodes from **the command line** (already covered)
3. Batch processing on compute nodes (already covered)



Graphical user interface (GUI)

1. Interactively graphically using the **X Window Server** (MobaXTerm, XQuartz)
2. Interactively graphically using **Open OnDemand (OOD)** on compute nodes (already shown)



Running GUI programs using the X Window System

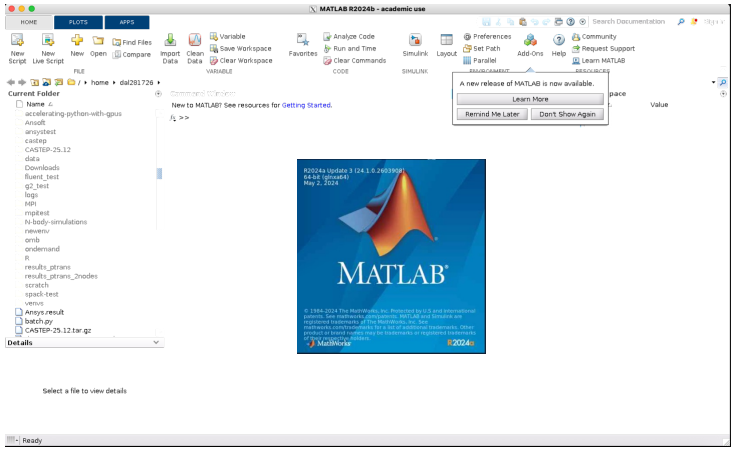
- MobaXterm and XQuartz offer free terminal with X11 server, tabbed SSH client, network tools and much more.

From MobaXterm (Windows):

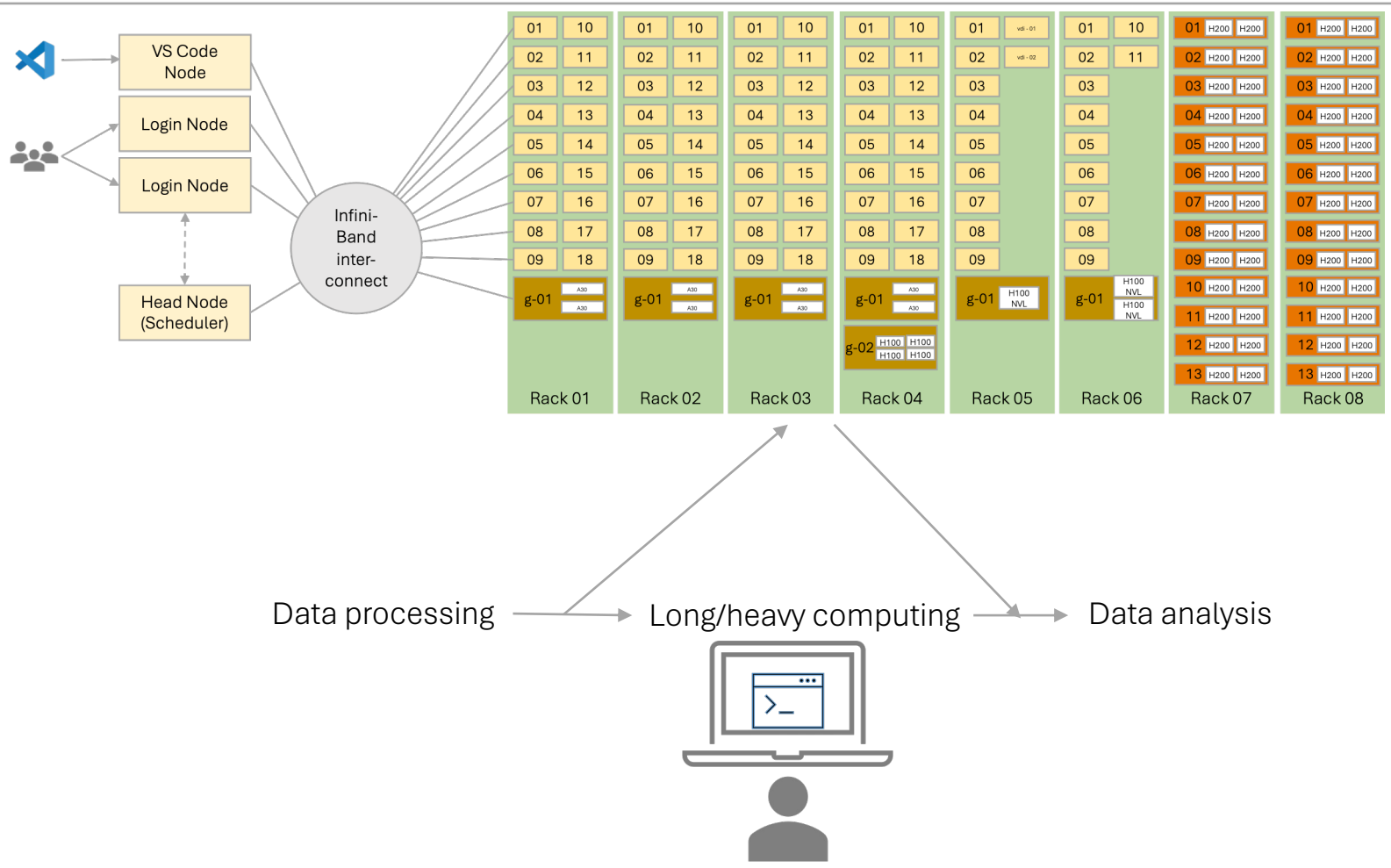
```
$ ssh -X <your-NetID>@juno.utdallas.edu
$ xclock # launch a GUI-based clock
$ module load matlab
$ matlab
```

From XQuartz (Mac):

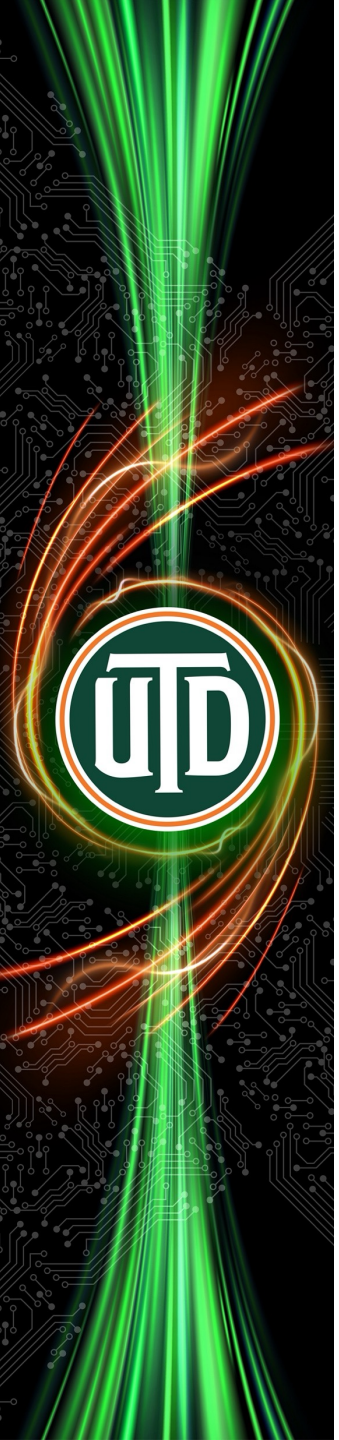
```
$ ssh -X <your-NetID>@juno.utdallas.edu
$ xclock # launch a GUI-based clock
$ module load matlab
$ matlab
```



HPC Offloading*



* Not all software/packages support this, and the setup varies



Compiling and running your own code

OpenMP/multithreaded b) MPI c) GPU d) Python

Refer to Juno user guide for more details: https://utdallas-hpc-juno-ug.readthedocs-hosted.com/en/latest/running-programs/slurm/

Serial programs

- Compile and run a serial C program

```
gcc -o crunch crunch.c
./crunch
```

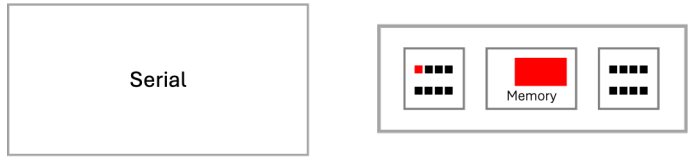
- **`crunch.c`** is the C language program or source file. Crunch is the compiled program file. After compilation, you can run **`crunch`** directly on the system.

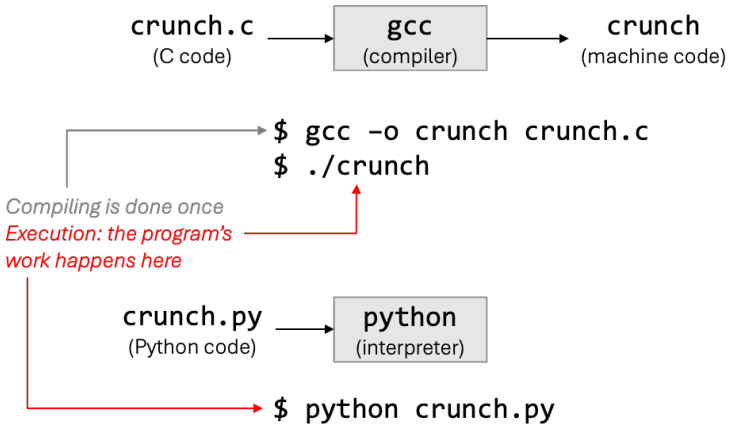
- Compile and run a serial Fortran program

```
gfortran -o crunch crunch.f90
./crunch
```

- Run a serial Python program

```
python crunch.py
```





Shared memory parallel programs

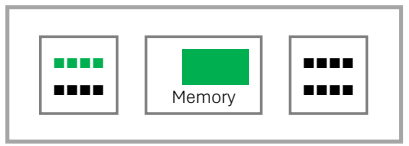
- Compile C program, run on 4 cores

```
gcc -fopenmp -o crunch_mt crunch_mt.c
export OMP_NUM_THREADS=4
./crunch_mt
```

- Compile Fortran program, run on 4 cores

```
gfortran -fopenmp -o crunch_mt crunch_mt.f90
export OMP_NUM_THREADS=4
./crunch_mt
```

Shared Memory Parallel
(OpenMP, pthreads)



Message passing parallel programs

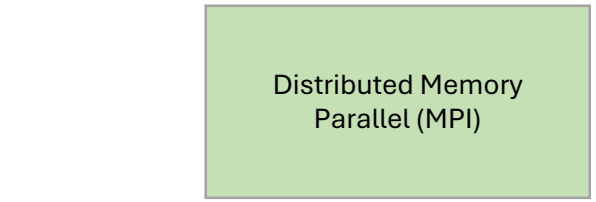
- Compile a distributed memory parallel C program and run it on 4 nodes

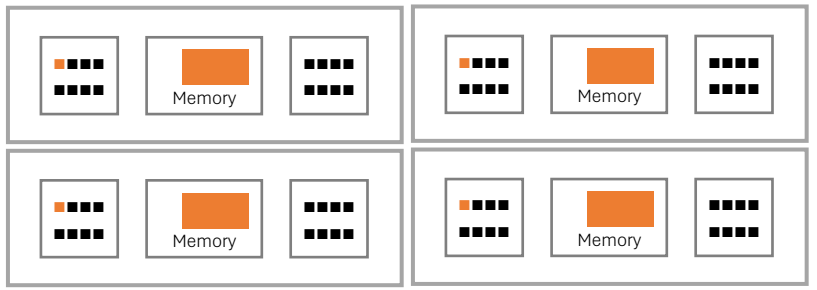
```
mpicc -o crunch_mpi crunch_mpi.c
mpirun –np 4 crunch_mpi
```

- Compile a distributed memory parallel Fortran program and run it on 4 nodes

```
mpif90 -o crunch_mpi crunch_mpi.f90
mpirun –np 4 crunch_mpi
```

Note: **`mpirun`** command must be run using SLURM batch system, it will not work on the login node. Why?



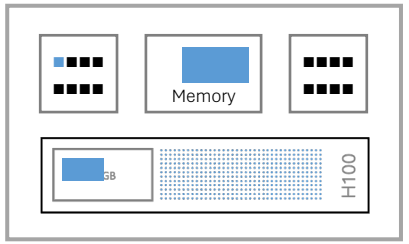


GPU accelerated programs

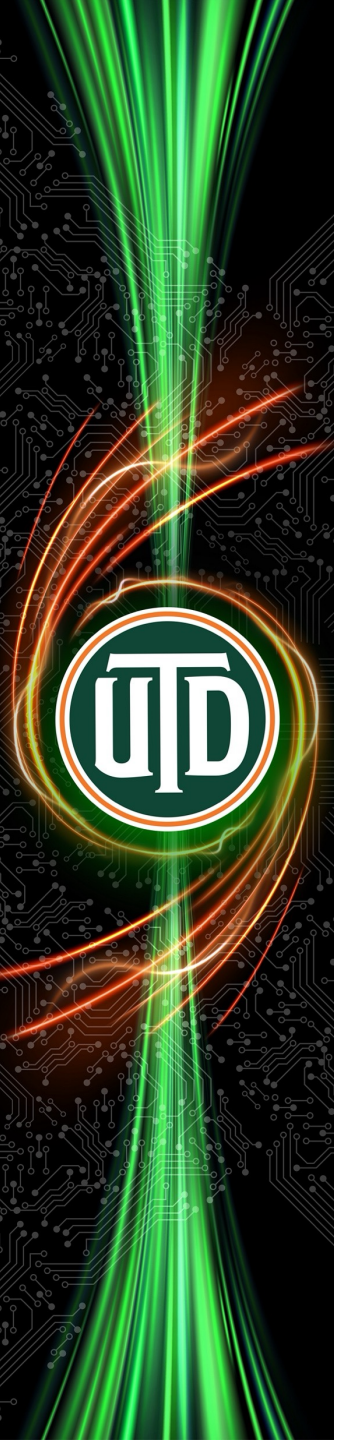
- To compile and run a C CUDA program that runs on a GPU using the thousands of small processors on the GPU to accelerate computation, use the following commands

```
module unload gnu12
module load cuda
nvcc -o diffusion_cuda diffusion_cuda.cu
./diffusion_cuda
```

GPU accelerated
(CUDA, OpenMP for GPU, OpenACC)



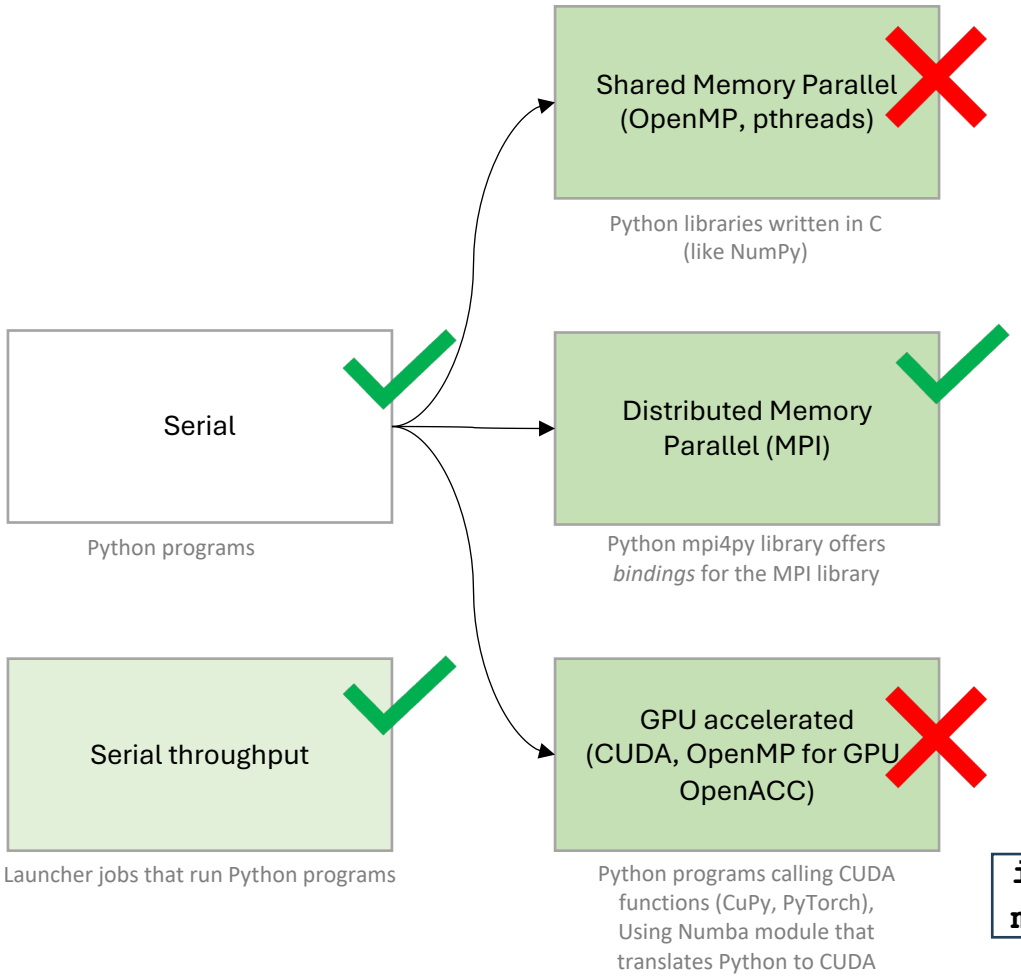
Note: The program must be run on a node with a GPU. If you run **`./diffusion_cuda`** on a node (e.g. login node) without a GPU, it may run but will produce incorrect results.



What makes Python programs different?

- Python is an interpreted language and usually is very slow
- Python programs use a lot of modules and libraries
  - Have to manage python, modules and libraries version matches/mismatches
    - Use Miniconda to install python “environments” in your directories
- Python can link with with functions written in C/C++ or Fortran
  - In fact, this is very common…
  - Shared memory parallelism is usually achieved using libraries code in OpenMP or pthreads
  - GPU acceleration is usually achieved using libraries coded in CUDA

Can pure Python programs run in these serial and parallel modes?



Python's Global Interpreter Lock (GIL) prevents Python threads from running on multiple cores, only one thread runs on once core at a time.

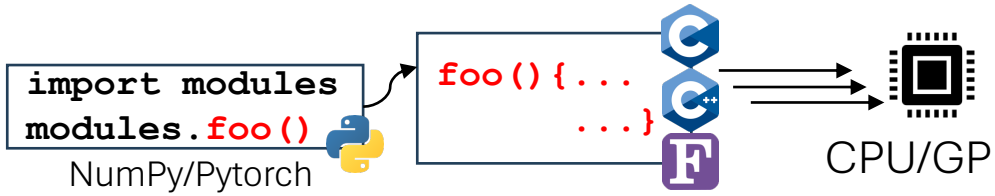
Python programs can call modules utilizing library functions written in C. Those C functions can run in shared memory or multiheaded parallel mode. NumPy is an example.

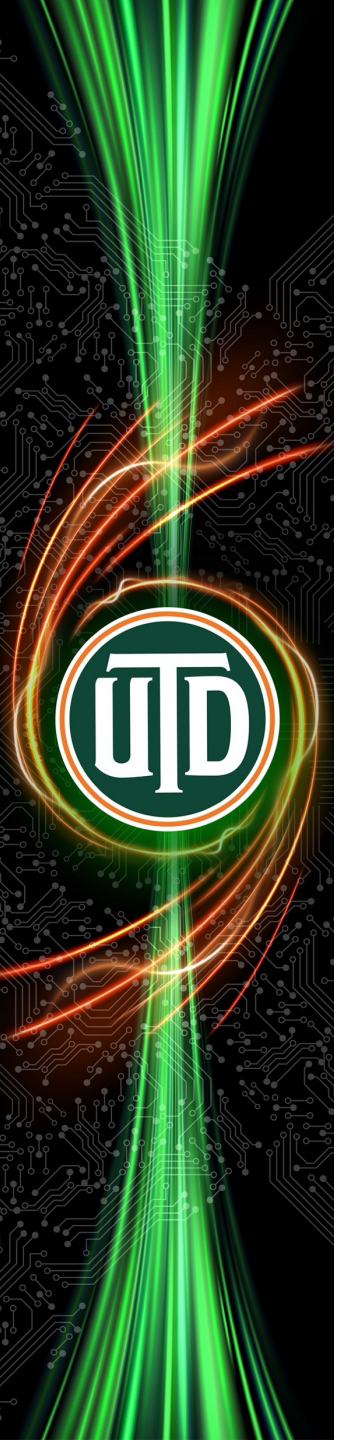
The mpi4py module provides Python bindings for the MPI library.

GPU programs are written using C and CUDA (C extensions and library).

The two most common ways Python programs use GPUs are:

- Use Python modules with functions written in C and CUDA (e.g., CuPy, PyTorch)
- Use Numba which translates Python code into CUDA code





Example of **miniconda** usage to install software (reference slide)

- Initialize and set up a **conda** environment

```
$ module load miniconda

$ conda create -n <env_name>
python=3.12 # This will be
installed in your /home directory

$ conda create -p <path_to_env>
python=3.12 # Recommended

$ conda init bash

$ source ~/.bashrc

$ conda activate <env_name>
```

- Installing software and packages

```
$ conda install scipy

$ conda install -c conda-forge
tensorflow

$ conda list

$ conda remove scipy

$ pip install --upgrade MDAnalysis
```

Hybrid shared memory and message passing programs

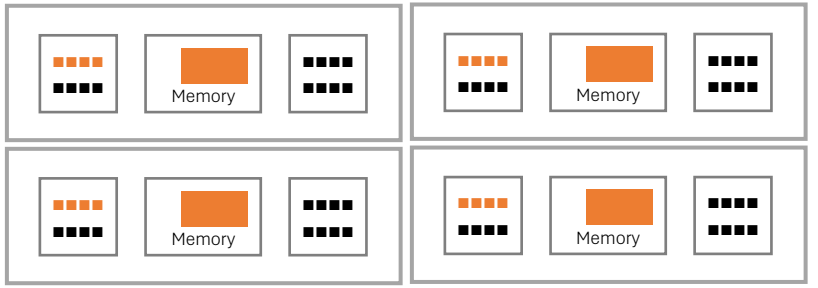
- To compile a C hybrid shared and distributed memory parallel program and run it on 4 cores/node on 4 nodes (total 16 cores)

```
mpicc -fopenmp -o crunch_mpi_mt
crunch_mpi_mt.c

export OMP_NUM_THREADS=4

mpirun –np 4 crunch_mpi_mt
```

Hybrid Shared and Distributed Memory Parallel

- To compile a Fortran hybrid shared and distributed memory parallel program and run it on 4 cores/node on 4 nodes (total 16 cores)

```
mpif90 -fopenmp -o crunch_mpi_mt
crunch_mpi_mt.f90

export OMP_NUM_THREADS=4

mpirun –np 4 crunch_mpi_mt
```



Note: **mpirun** command must be run using SLURM job scheduler, it will not work on the login node.

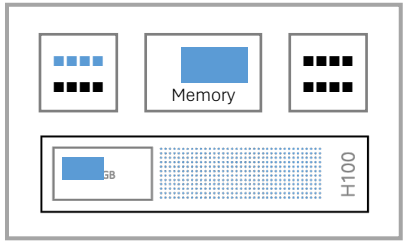
For reference

Hybrid shared memory and GPU accelerated

- To compile a C hybrid shared memory parallel and GPU-accelerated program and run it on 4 cores

```
module unload gnu14

module load cuda

nvcc -Xcompiler -fopenmp
hybrid_omp_cuda.cu -o hybrid_omp_cuda

export OMP_NUM_THREADS=4

./hybrid_omp_cuda
```

Hybrid Shared Memory Parallel and GPU-accelerated



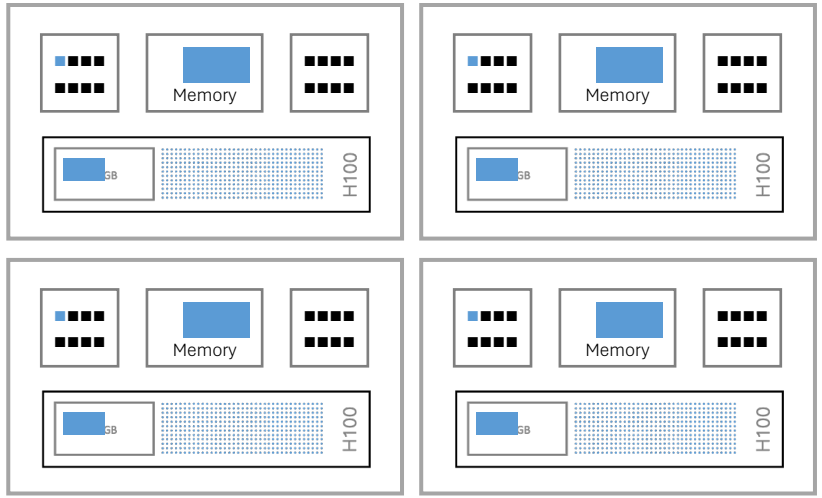
For reference

Hybrid message passing and GPU accelerated

- To compile a C hybrid shared and distributed memory parallel and GPU-accelerated program and run it on 4 cores

```
module unload gnu14

module load cuda

nvcc -Xcompiler -ccbin=mpicxx
hybrid_mpi_cuda.cpp -o hybrid_mpi_cuda

mpirun -n 4 ./hybrid_mpi_cuda
```

Hybrid Distributed Memory Parallel and GPU acceleration



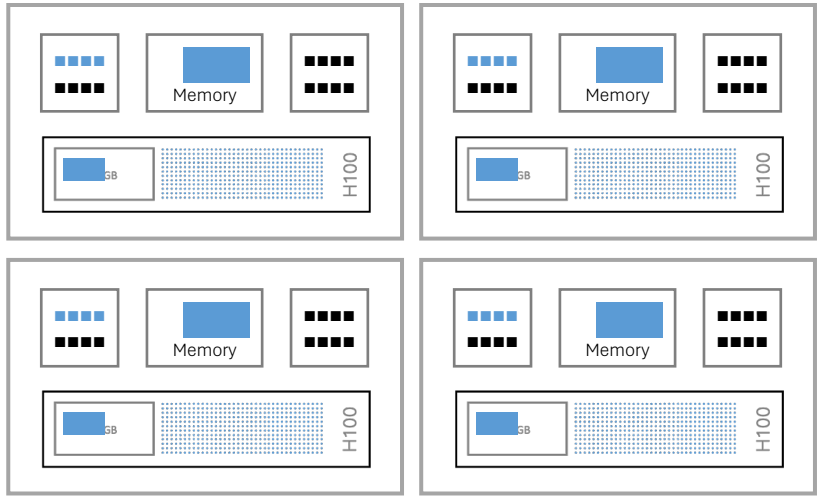
For reference

Hybrid “everything” programs

- To compile a C hybrid shared and distributed memory parallel and GPU-accelerated program and run it on 4 cores

```
module unload gnu14

module load cuda

nvcc -Xcompiler -fopenmp -ccbin=mpicxx
hybrid_openmp_mpi_cuda.cpp -o
hybrid_openmp_mpi_cuda

export OMP_NUM_THREADS=4

mpirun -n 4 ./hybrid_openmp_mpi_cuda
```

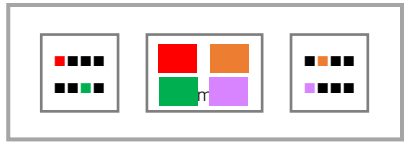
Hybrid Distributed Memory Parallel and GPU acceleration



For reference

High throughput workloads using Launcher

High throughput



```
# tasklist.txt
python myscript.py input_1.txt output_1.txt
python myscript.py input_2.txt output_2.txt
python myscript.py input_3.txt output_3.txt
...
python myscript.py input_100.txt output_100.txt
```

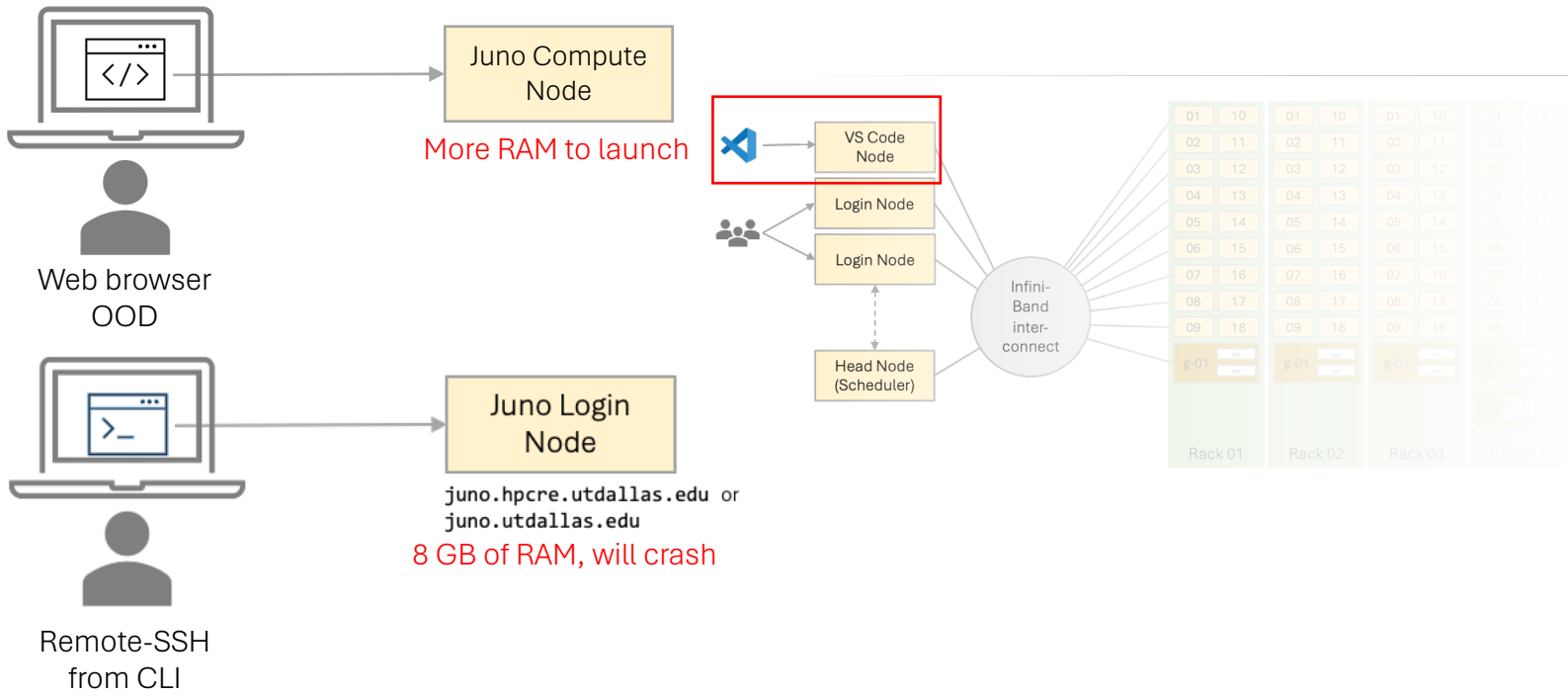
```
# job.sh
#!/bin/bash
#SBATCH --job-name=py_launcher
#SBATCH --nodes=2
#SBATCH --partition=normal
#SBATCH --ntasks-per-node=64
#SBATCH --time=00:30:00
#SBATCH --output=py_launcher.out
#SBATCH --error=py_launcher.err

# Load required modules
module load launcher

# Define paths
export LAUNCHER_WORKDIR=$PWD
export LAUNCHER_JOB_FILE=$PWD/tasklist.txt

export LAUNCHER_PPN=64   # tasks per node
export LAUNCHER_NHOSTS=2
echo "Starting high-throughput Python tasks..."
$LAUNCHER_DIR/paramrun
echo "All tasks completed."
```

```
(base) [dal281726@juno-l-02 ~]$ pwd
/home/dal281726
(base) [dal281726@juno-l-02 ~]$ ls
input_1.txt input_2.txt input_3.txt input_4.txt input_5.txt input_6.txt input_7.txt
input_8.txt input_9.txt ... input_100.txt job.sh myscript.py tasklist.txt
(base) [dal281726@juno-l-02 ~]$ sbatch job.sh
Submitted batch job 2540797
```

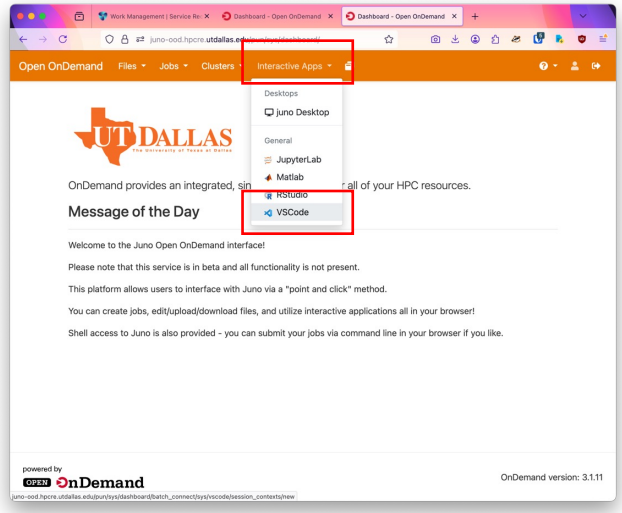
For reference

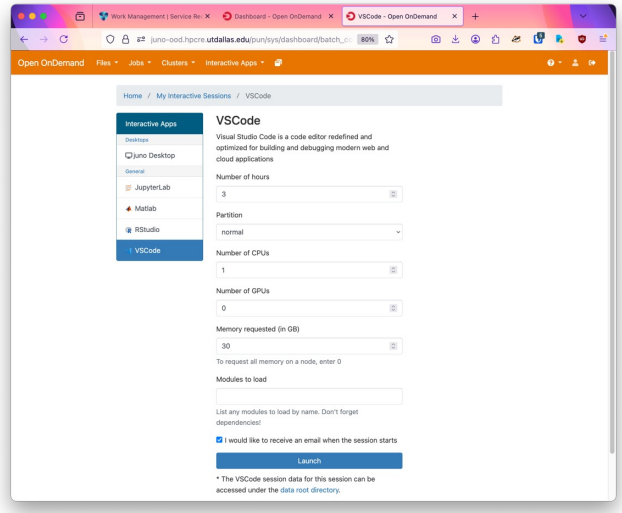
IDE: VS Code

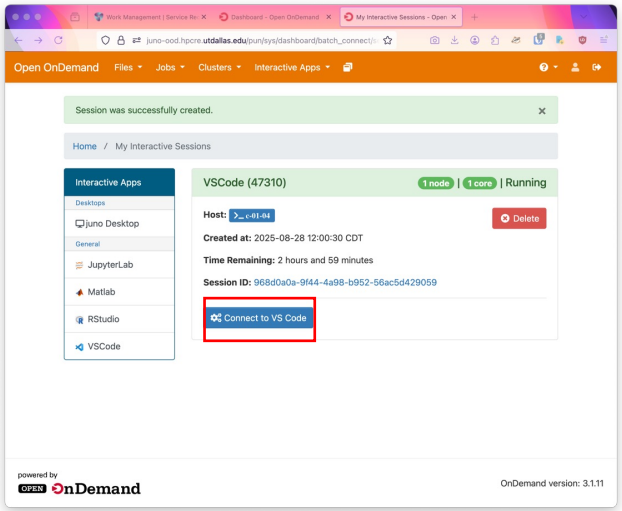
- Running on Open OnDemand
- Running on your workstation using Remote-SSH extension (not recommended)
  - A limit of 8GB of RAM on Juno login nodes, use **`juno-vscode.utdallas.edu`** instead)

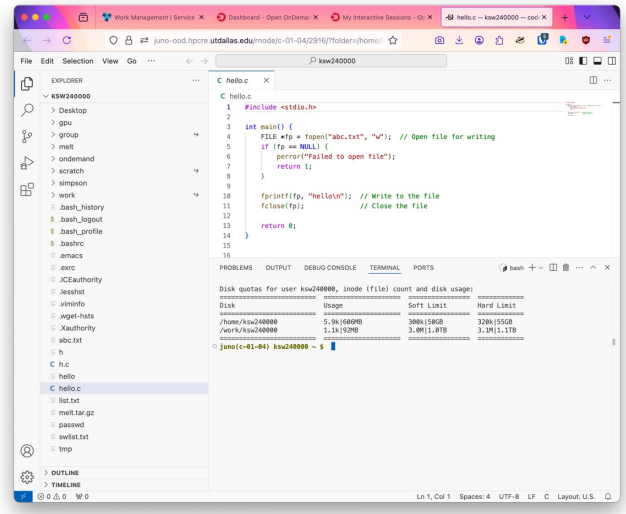


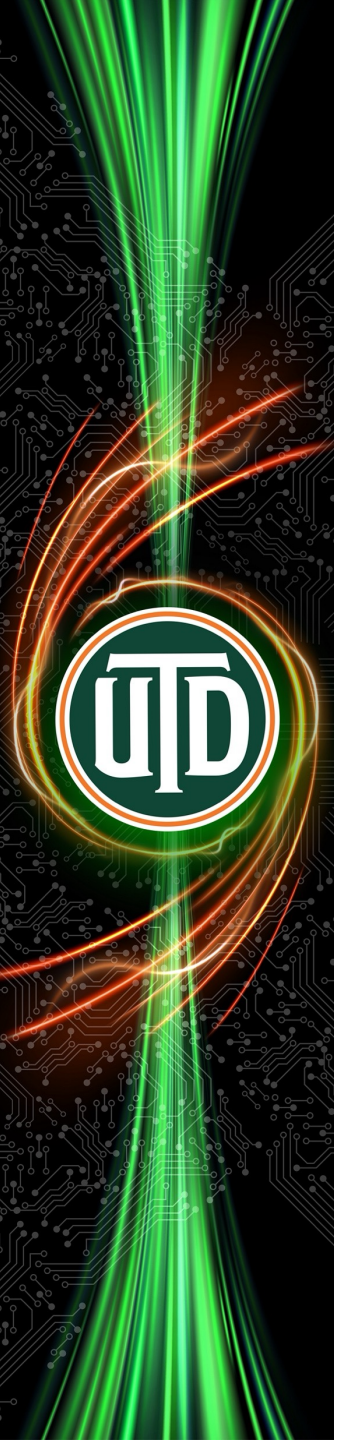
VSCode on Open OnDemand











Running pre-packaged programs

- Matlab (CLI, GUI)
- Gaussian
- Fluent (CLI, GUI)
- Containers

Refer to Juno user guide for more details: https://utdallas-hpc-juno-ug.readthedocs-hosted.com/en/latest/running-programs/common-programs/

MATLAB

- Command line interface

1. Running interactively: request a compute node and start an interactive session, then load and run the script

```
$ salloc -p normal --mem=2GB
$ srun --pty bash
$ module load matlab
$ matlab -nodisplay -nosplash -nodesktop -r
"run('path/to/your/script.m'); exit;"
```

2. Running as a batch job:

```
#slurm.sh
$ module load matlab
$ matlab -nodisplay -nosplash -nodesktop -r
"run('path/to/your/script.m'); exit;"
```

For reference

MATLAB

- With a graphical user interface (MobaXterm, XQuartz):

Make sure that you logged in to Juno with the -X option:

```
$ ssh -X <netID>@juno.utdallas.edu
```

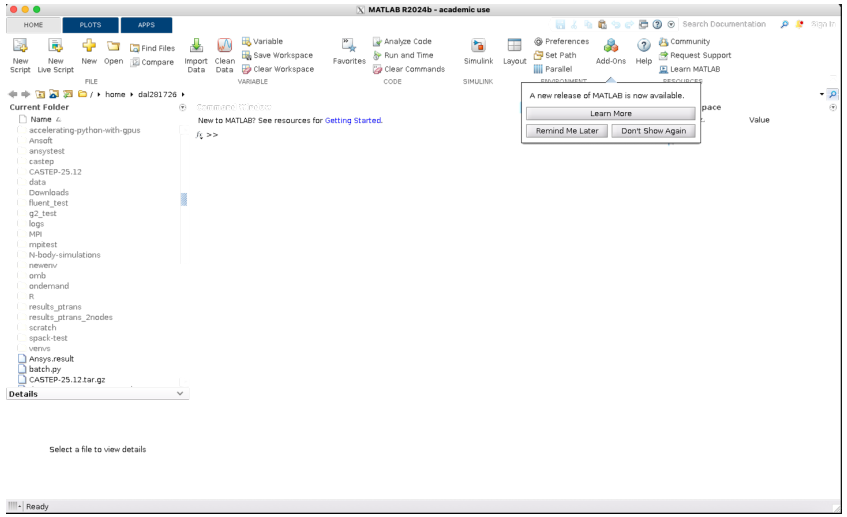
Request a compute node and start an interactive session

```
$ salloc -p normal --mem=2GB
$ squeue --me #to check which node is assigned to you
$ ssh -X c-04-01
```

Load the module and run

```
$ module load matlab
$ matlab
```

```
(base) [dal281726@juno-l-01 ~]$ salloc -p normal --mem=2GB
salloc: Granted job allocation 69229
Disk quotas for user dal281726, inode (file) count and disk usage:
========================  ====================  ================  ============
Disk                      Usage                 Soft Limit        Hard Limit
========================  ====================  ================  ============
/home/dal281726           174k|42GB             300k|50GB         320k|55GB
/work/dal281726           16k|2.9GB             3.0M|1.0TB        3.1M|1.1TB
========================  ====================  ================  ============
(base) [dal281726@juno-l-01 ~]$ squeue --me
             JOBID    PARTITION                NAME      USER ST       TIME  NODES NODELIST(REASON)
             69082 new-cpu-test                Abu4 dal281726  R   18:09:59      1 c-06-01
             69229       normal         interactive dal281726  R       0:04      1 c-04-01
             69224       normal         interactive dal281726  R      17:29      1 c-04-01
(base) [dal281726@juno-l-01 ~]$ ssh -X c-04-01
Warning: Permanently added 'c-04-01' (ED25519) to the list of known hosts.
Disk quotas for user dal281726, inode (file) count and disk usage:
========================  ====================  ================  ============
Disk                      Usage                 Soft Limit        Hard Limit
========================  ====================  ================  ============
/home/dal281726           174k|42GB             300k|50GB         320k|55GB
/work/dal281726           16k|2.9GB             3.0M|1.0TB        3.1M|1.1TB
========================  ====================  ================  ============
(base) [dal281726@c-04-01 ~]$ module load matlab
(base) [dal281726@c-04-01 ~]$ matlab
MATLAB is selecting SOFTWARE rendering.
```



For reference

Gaussian

- Command line interface

1. Running interactively: request a compute node and start an interactive session, then load and run the script

```
$ salloc -p normal --mem=2G
$ srun --pty bash
$ module load gaussian
$ g16 input.com > output.log
```

2. Running as a batch job:

```
#slurm.sh
$ module load gaussian
$ g16 input.com > output.log
```

Gaussian automatically uses **`~/scratch`** to store **`.rwf`** files during batch jobs, you don't need to specify the path for these files.

For reference

Ansys Fluent

- Command line interface

1. Running interactively: request a compute node and start an interactive session, then load and run the script

```
$ salloc -p normal --mem=2GB
$ srun --pty bash
$ module load ansys/2025R1
$ fluent 2d -g -i sample.jou
```

2. Running as a batch job:

```
#slurm.sh
$ module load ansys/2025R1
$ fluent 2d -g -i sample.jou
```

For reference

Ansys Fluent

- With a graphical user interface (MobaXterm, XQuartz):

Make sure that you logged in to Juno with the -X option:

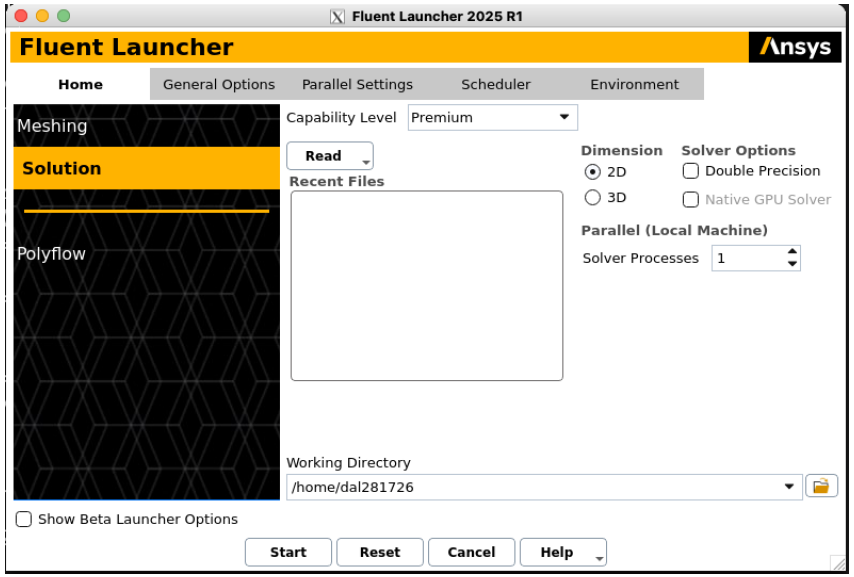
```
$ ssh -X <netID>@juno.utdallas.edu
```

Request a compute node and start an interactive session

```
$ salloc -p normal --mem=2GB
$ squeue --me #to check which node is assigned to you
$ ssh -X c-04-01
```

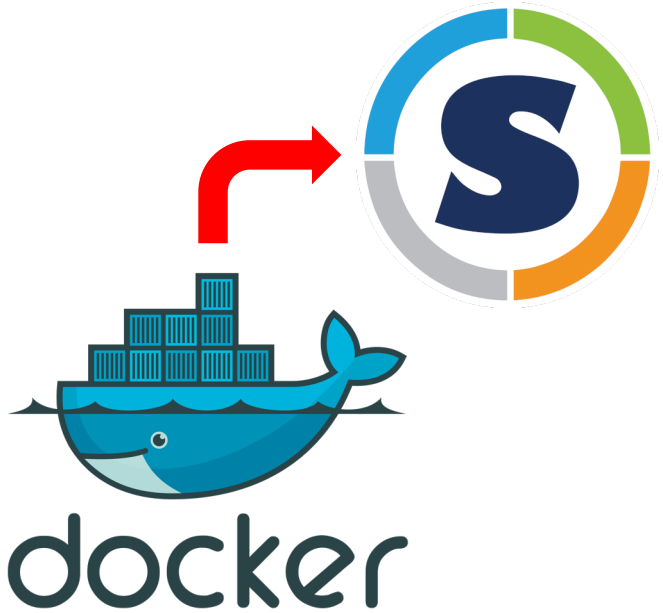
Load the module and run

```
$ module load ansys/2025R1
$ fluent
```



For reference

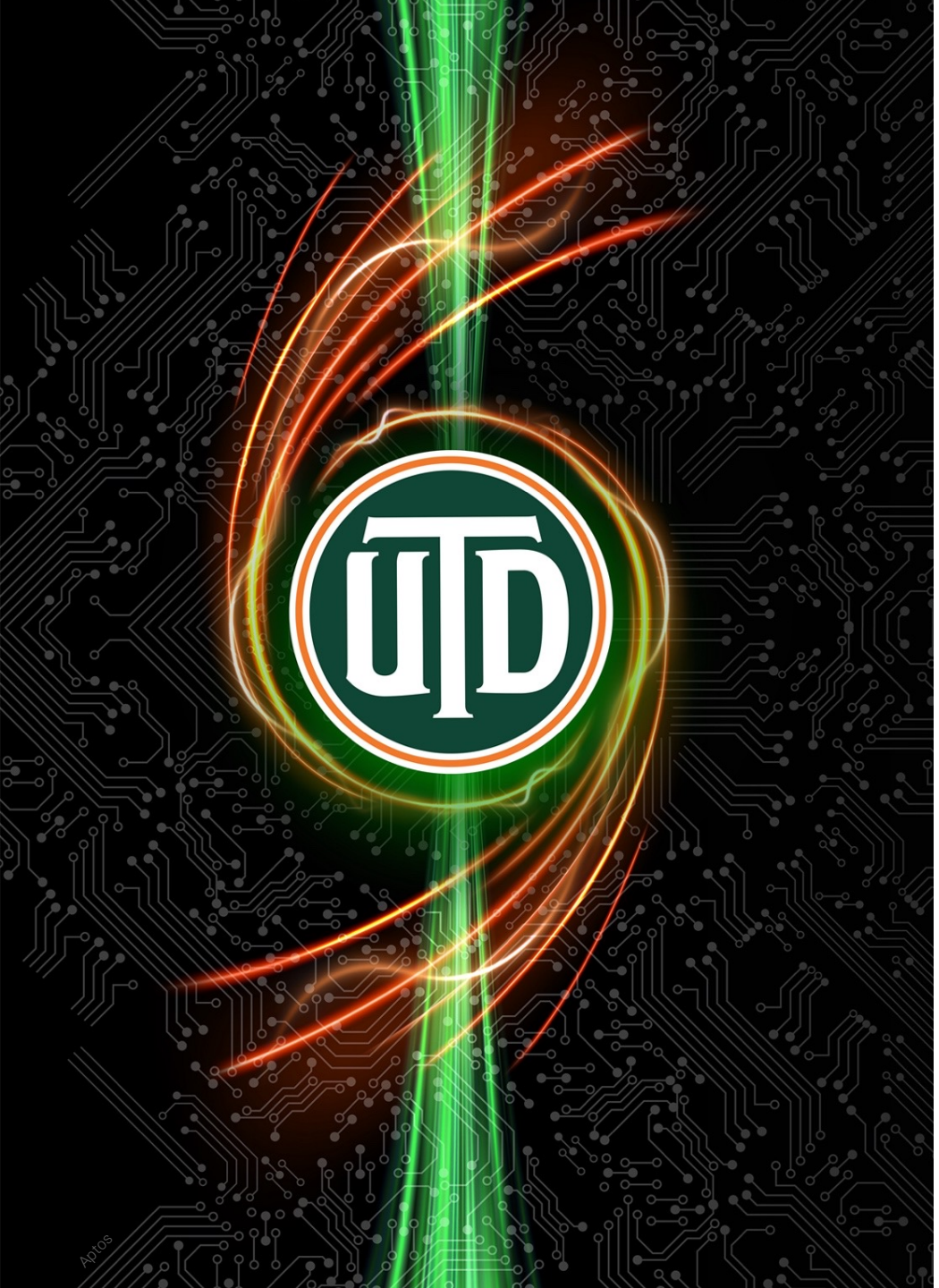
Containers



- Docker normally runs as the root user, which gives it full control of the system.
- On shared HPC clusters, that's a big security risk, because one user's container could potentially affect others or the system itself.
- You can convert your Docker images to Singularity/Apptainer images and run on Juno

```
#!/bin/bash
#SBATCH --job-name=apptainer_test
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --time=00:10:00
#SBATCH --output=apptainer_test.out
#SBATCH --error=apptainer_test.err

module load apptainer
# Define paths
CONTAINER=/path/to/my_container.sif
WORKDIR=$PWD

echo "Running Python script inside Apptainer container..."
apptainer exec \
    --bind $WORKDIR:/work \
    $CONTAINER \
    python /work/myscript.py arg1 arg2
echo "Done!"
```

For reference



Thank you

Questions? Comments? Feedback?

HPC@UTD